

CSE 207: Data Structures and Algorithms-II (Part 1)

Instructor
Dr Md Monirul Islam

Finding Minimum Spanning Tree

GENERIC-MST(G, w)

```
1   $A = \emptyset$ 
2  while  $A$  does not form a spanning tree
3      find an edge  $(u, v)$  that is safe for  $A$ 
4       $A = A \cup \{(u, v)\}$ 
5  return  $A$ 
```

A is a subset of some MST

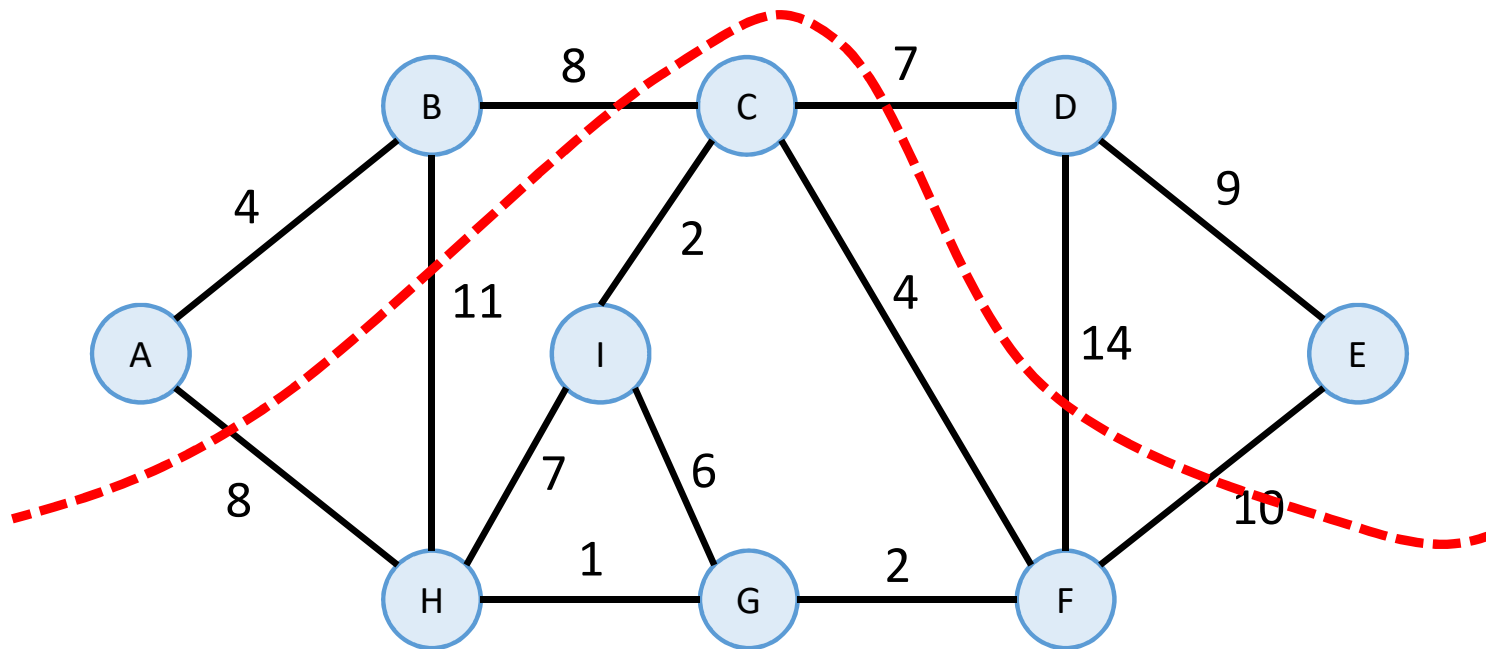
Grows gradually

One **safe edge** is added in each iteration

After termination, A is an MST

Finding a safe edge for a growing MST

Cut in graph $G=(V, E)$: a partition of V : $(S, V-S)$

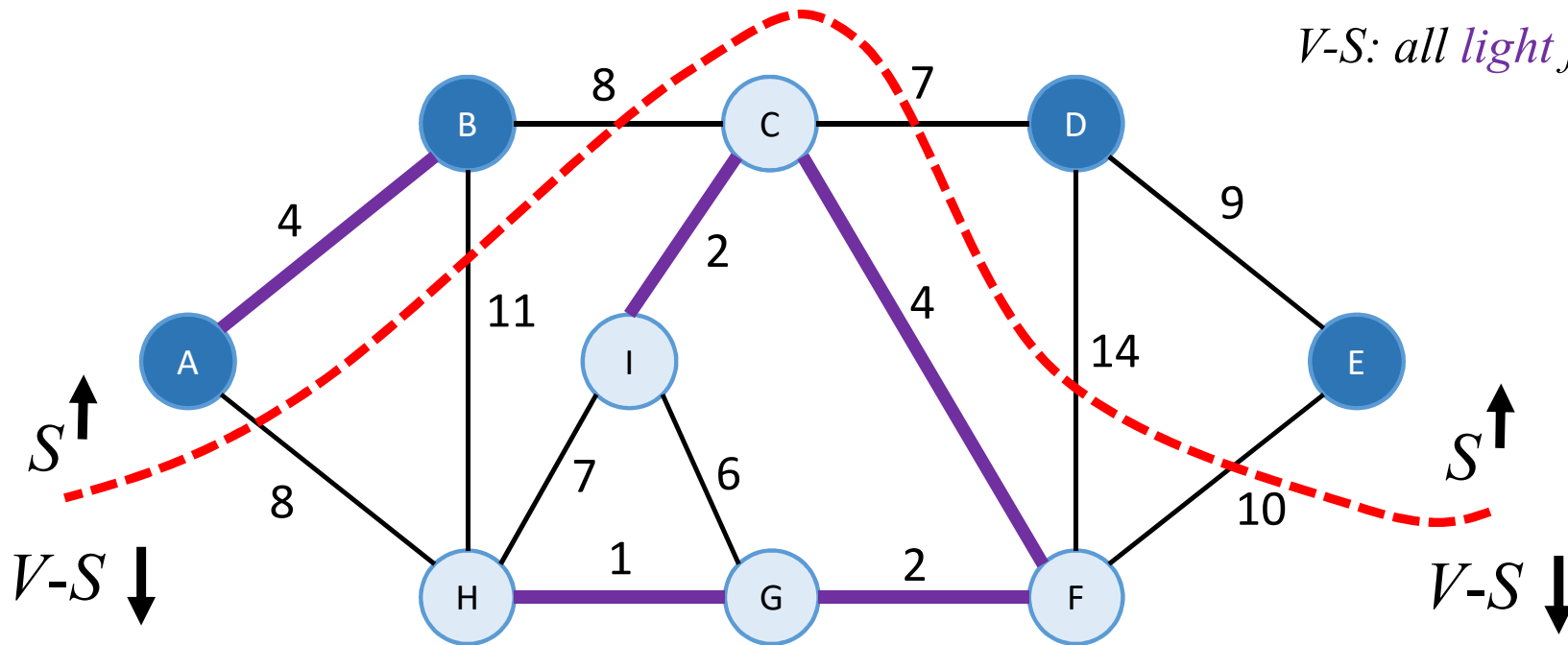


Finding a safe edge for a growing MST

Cut in graph $G=(V, E)$: a partition of V : $(S, V-S)$

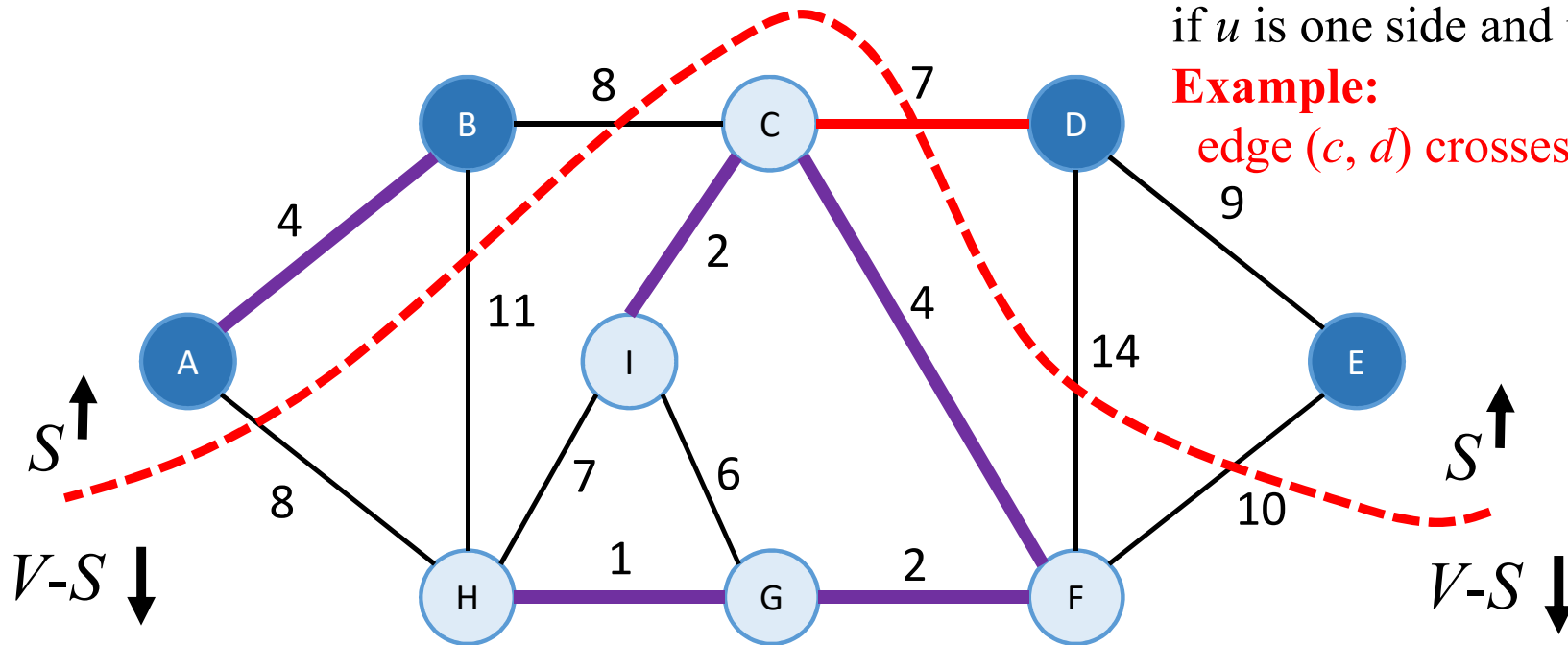
S : all *dark* filled vertices

V - S : all *light* filled vertices



Finding a safe edge for a growing MST

Cut in graph $G=(V, E)$: a partition of V : $(S, V-S)$



(u, v) crosses the cut

if u is on one side and v is on the other

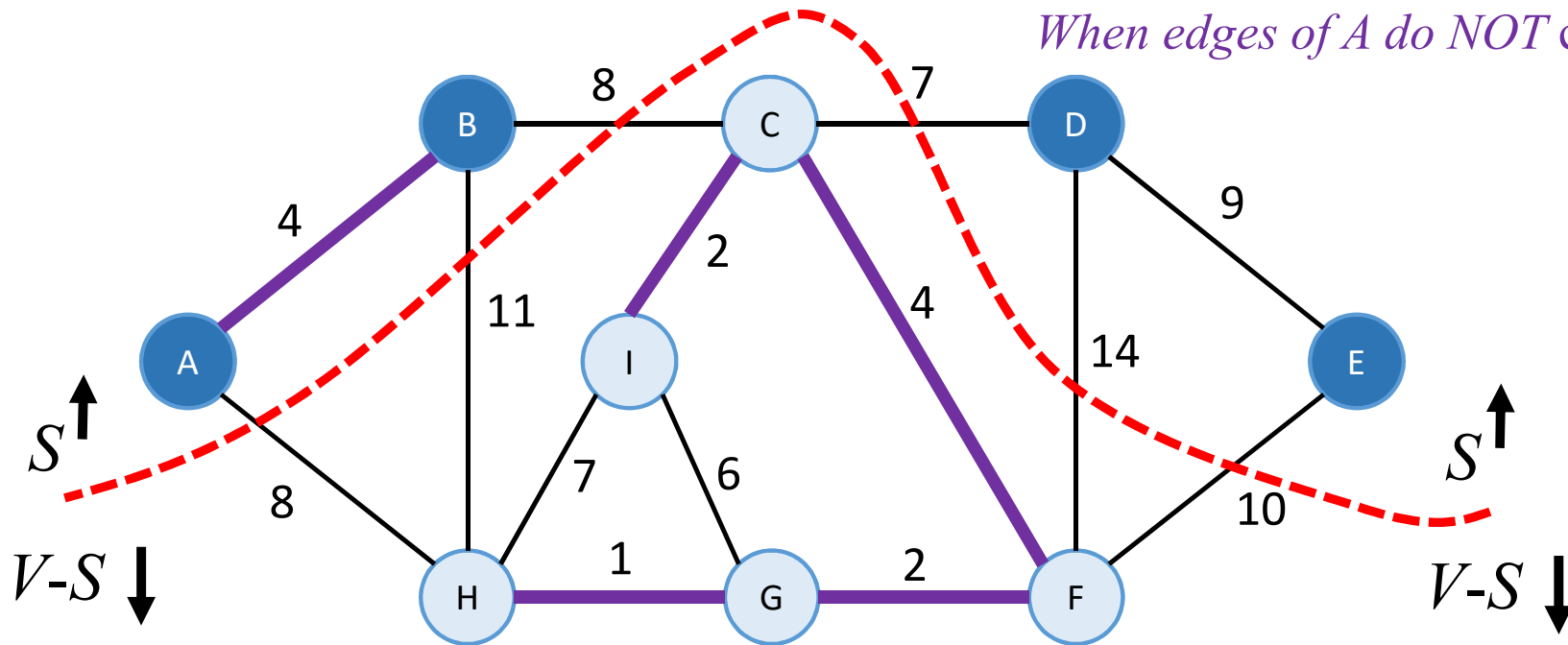
Example:

edge (c, d) crosses the cut

Finding a safe edge for a growing MST

Let A is the set of thick edges

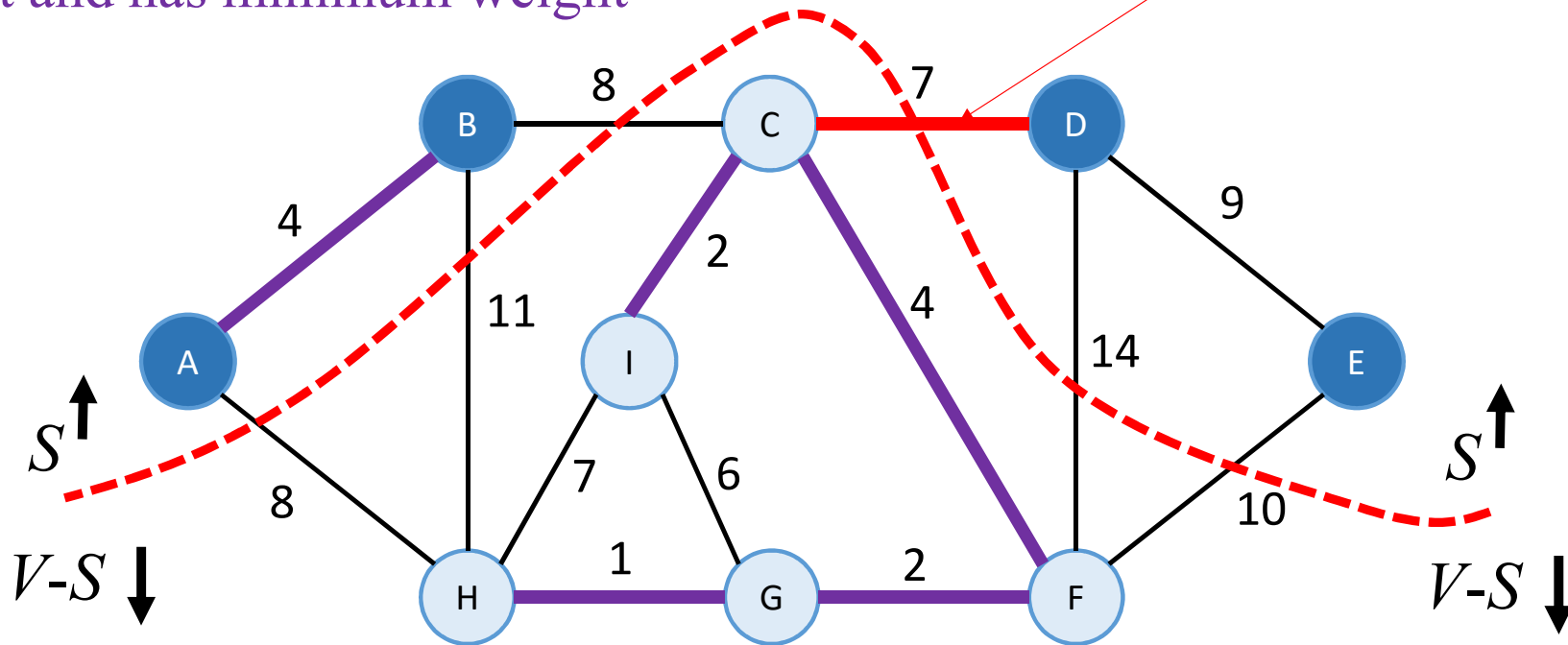
The cut $(S, V-S)$ respects A
When edges of A do NOT cross the cut



Finding a safe edge for a growing MST

Light edge (u, v) crosses the cut and has minimum weight

The edge (c, d) is a light edge



Finding a safe edge for a growing MST

Theorem 23.1:

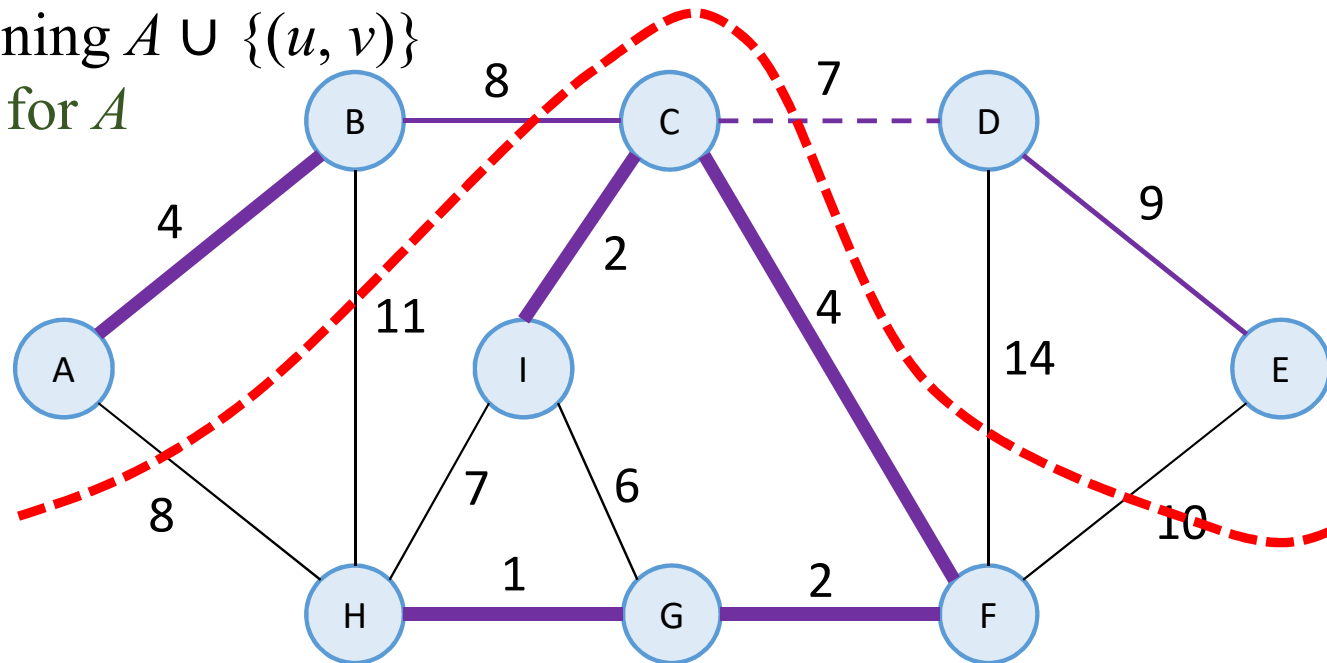
Let A be a set of edges, and consider a cut that respects A .

there is an MST containing A .

(u, v) be a light edge.

Then there is an MST containing $A \cup \{(u, v)\}$

In other words, (u, v) is safe for A



Finding a safe edge for a growing MST

Theorem 23.1:

Let A be a set of edges, and consider a cut that respects A .

there is an MST containing A .

(u, v) be a light edge.

Then there is an MST containing $A \cup \{(u, v)\}$

Explanation:

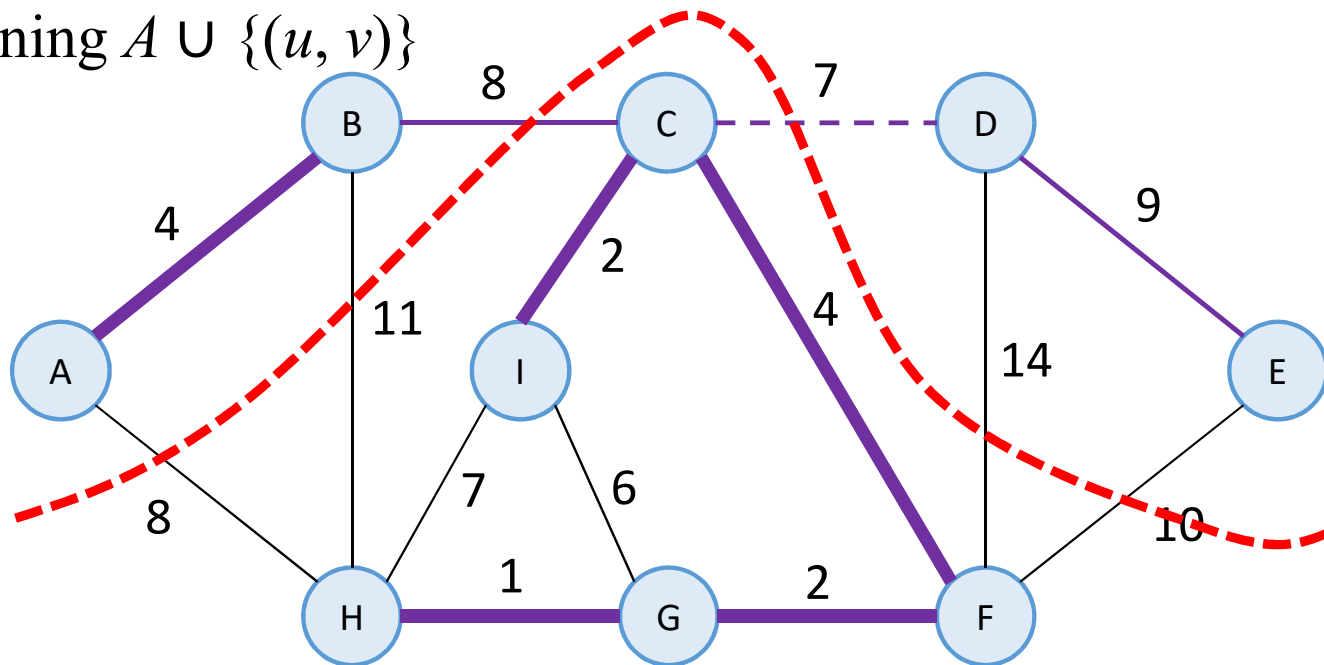
A = set of **thick** edges

T = an MST: all violet edges

$A \subset T$

(c, d) is a **light** edge.

Then, $A \cup \{(c, d)\} \subset T$



Proof of Theorem 23.1

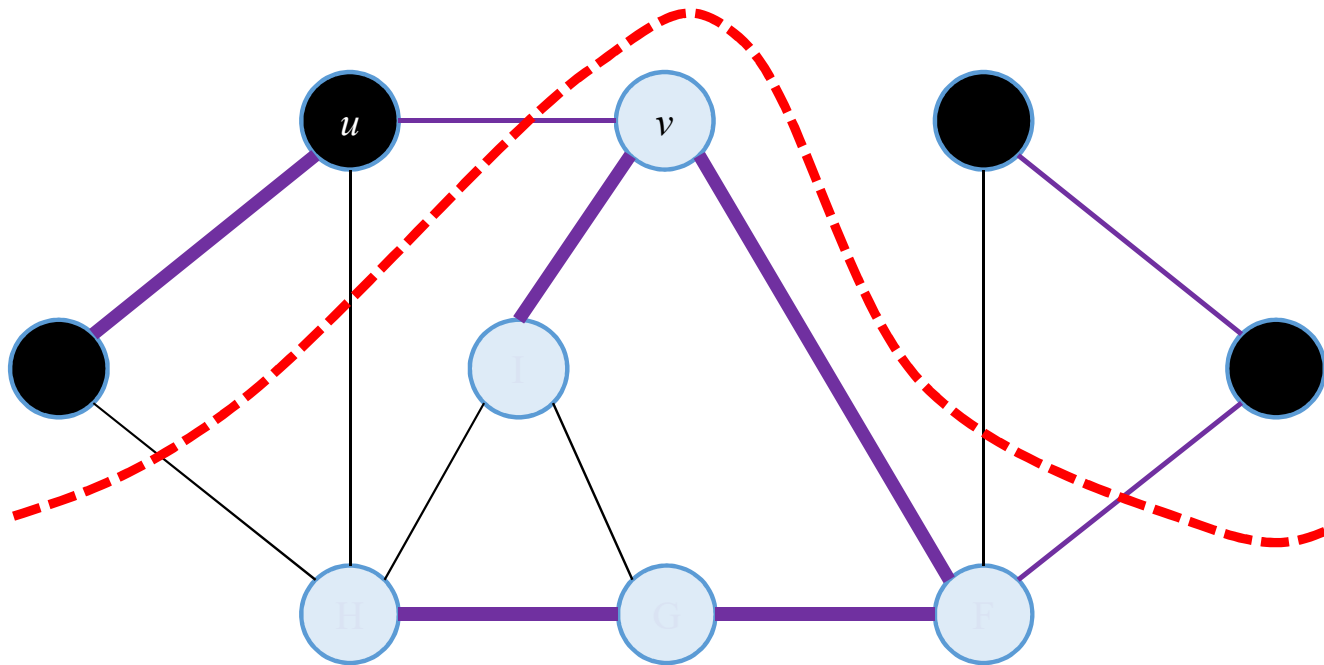
T = an MST

A = set of **thick** edges

Assume, $A \subset T$

S = black vertices

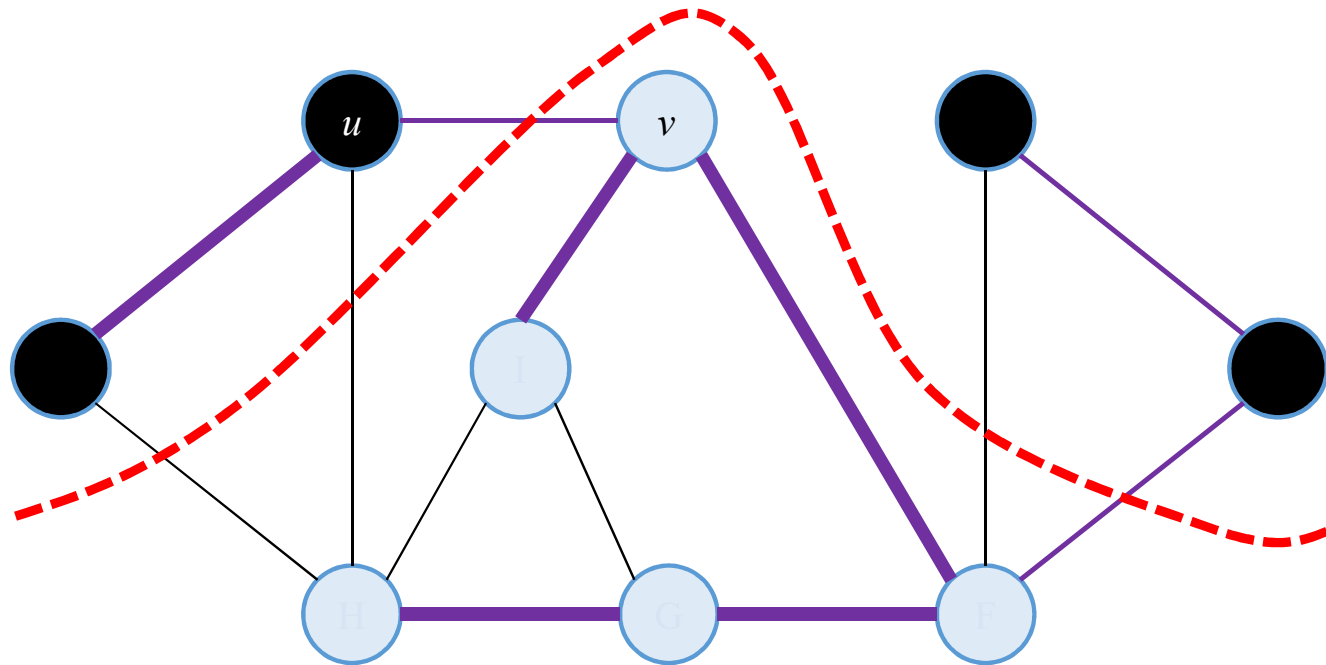
Cut $(S, V-S)$ respects A



Proof of Theorem 23.1

Let (u, v) be a light edge
but $(u, v) \notin A$.

Why?



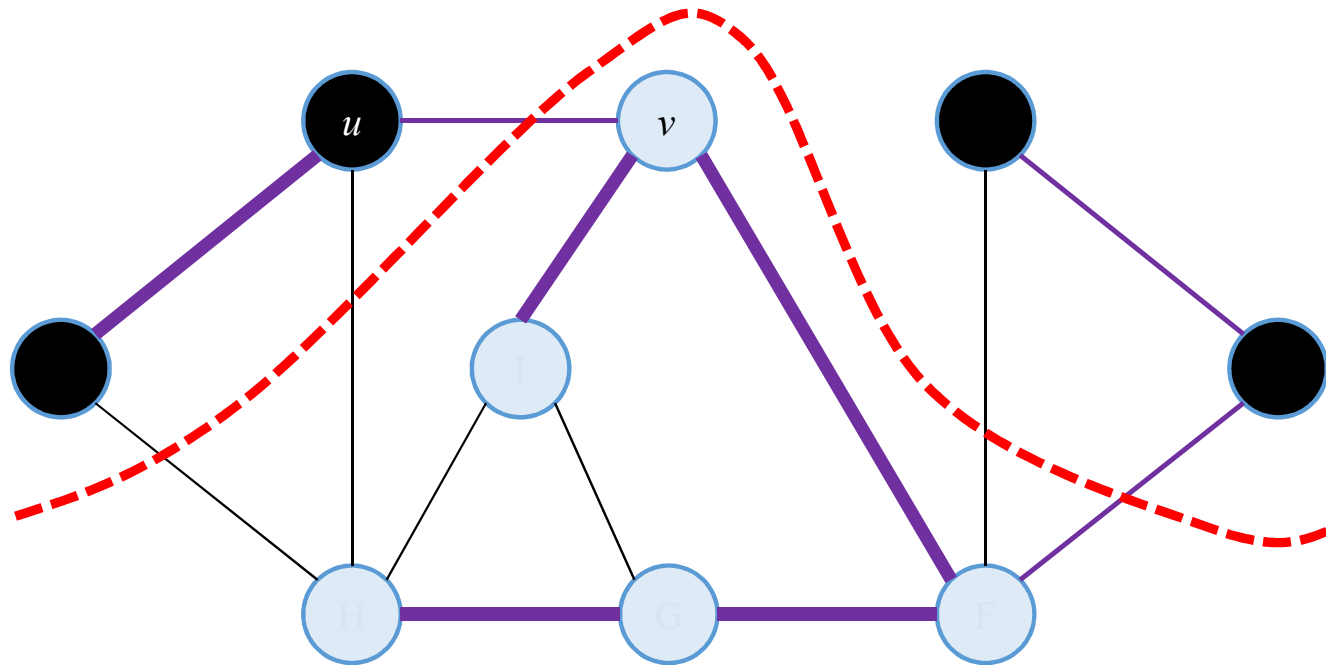
Proof of Theorem 23.1

Let (u, v) be a light edge **but** $(u, v) \notin A$.

If $(u, v) \in T$, we are done!

Then, $A \cup \{(u, v)\} \subset T$

(u, v) is safe for A .



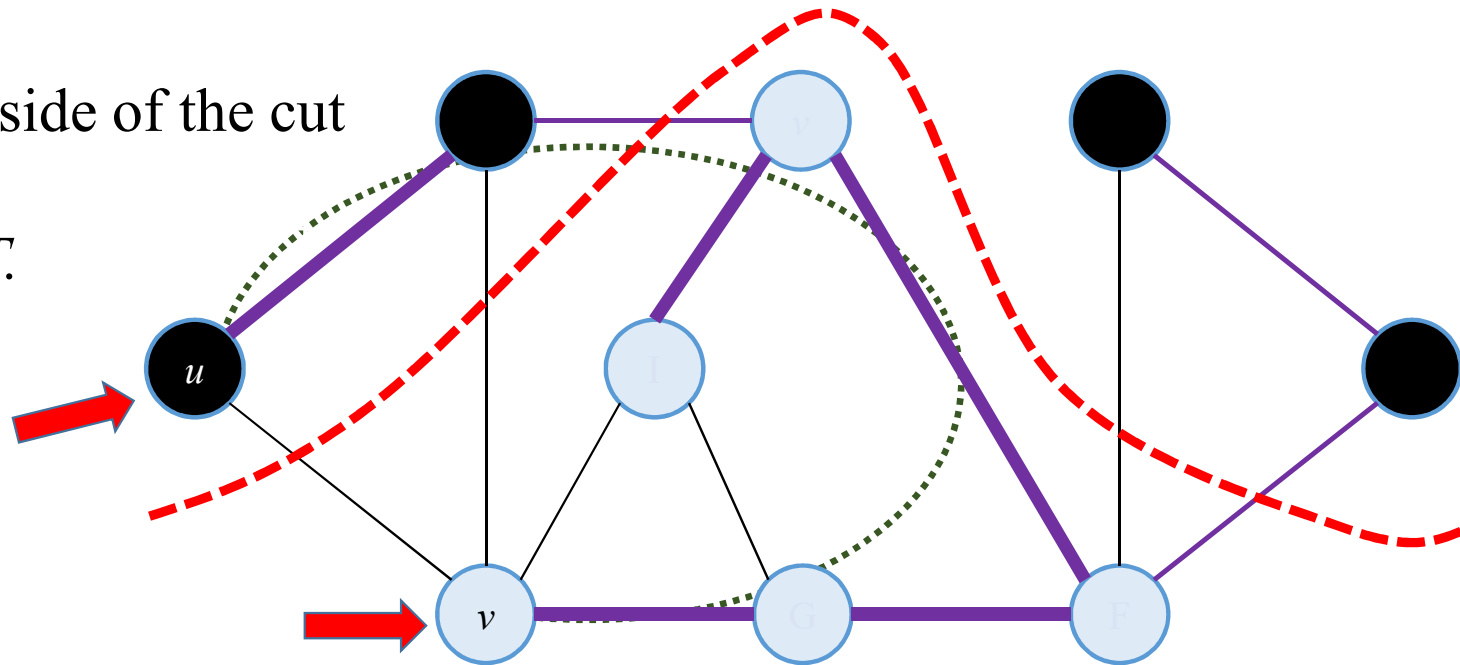
Proof of Theorem 23.1

(u, v) is a light edge **but** $(u, v) \notin A$.

If $(u, v) \notin T$, we make another MST, T'

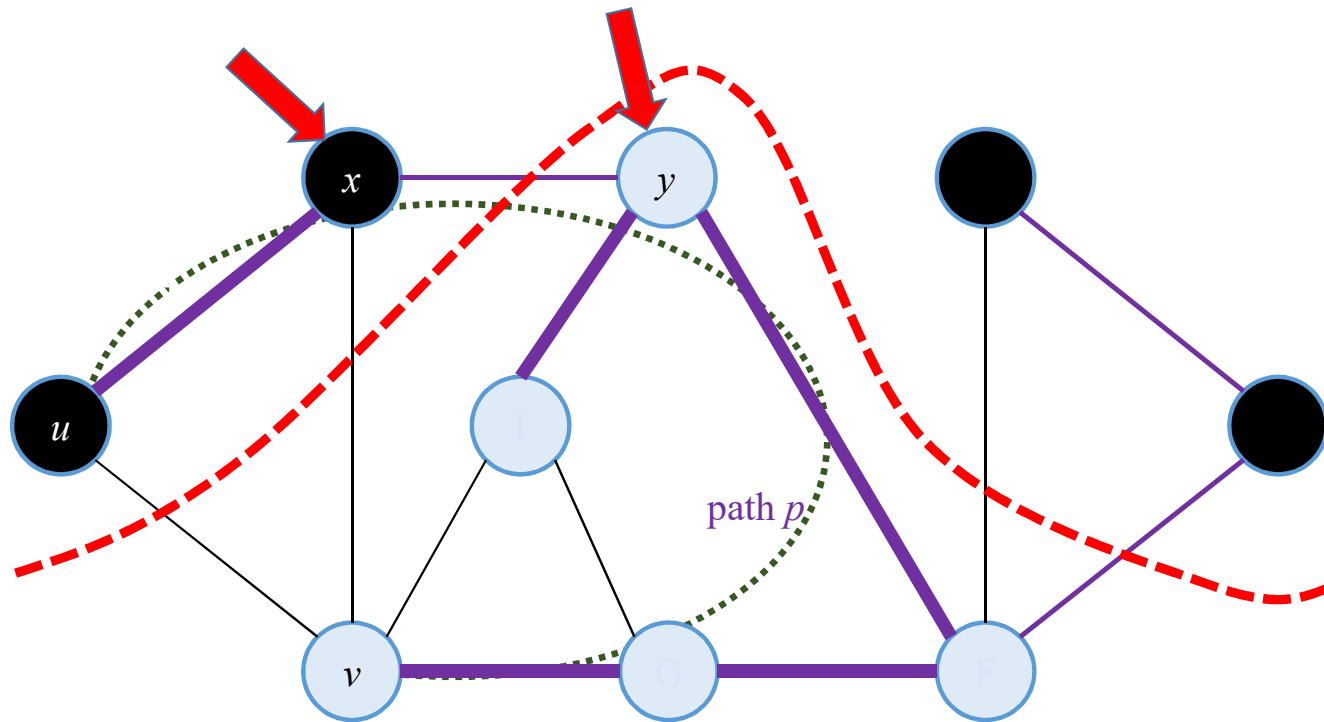
u and v are in different side of the cut

u and v make cycle in T .



Proof of Theorem 23.1

The simple **path p** from u to v in T must have an **edge (x, y)** that **crosses the cut**



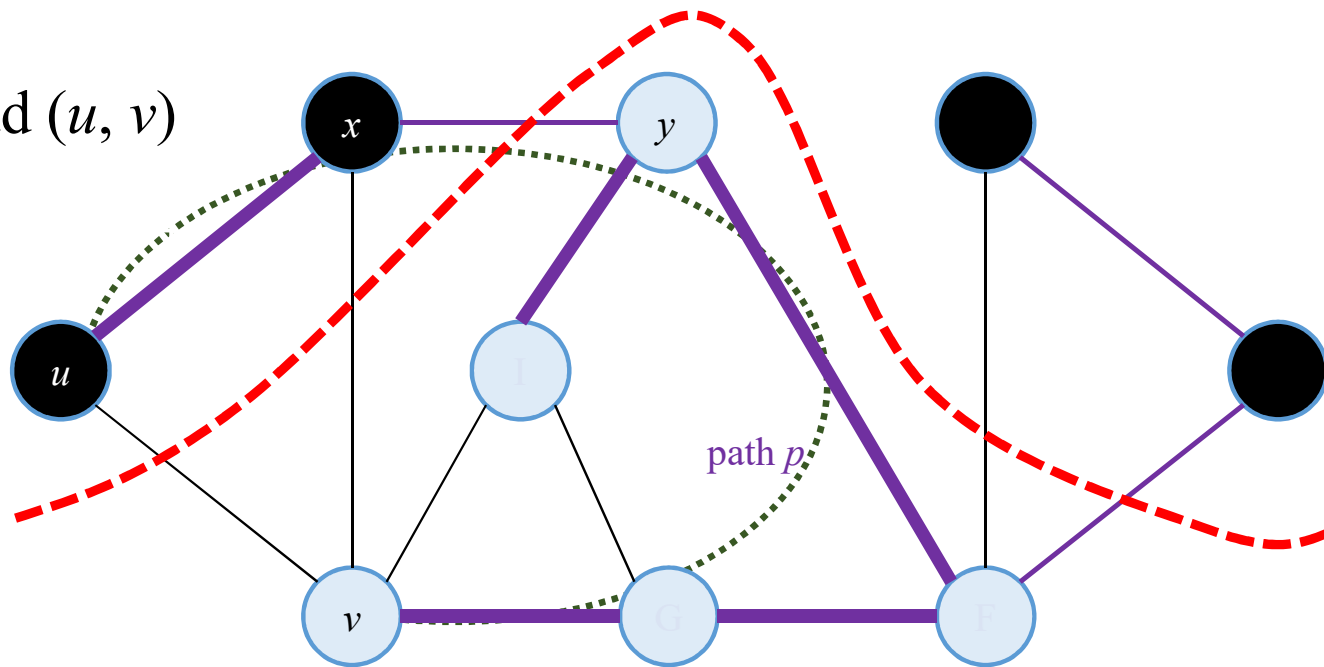
Proof of Theorem 23.1

The simple **path p** from u to v in T must have an **edge (x, y)** that **crosses the cut**

We guarantee, $(x, y) \notin A$.

Remove (x, y) from T and add (u, v)

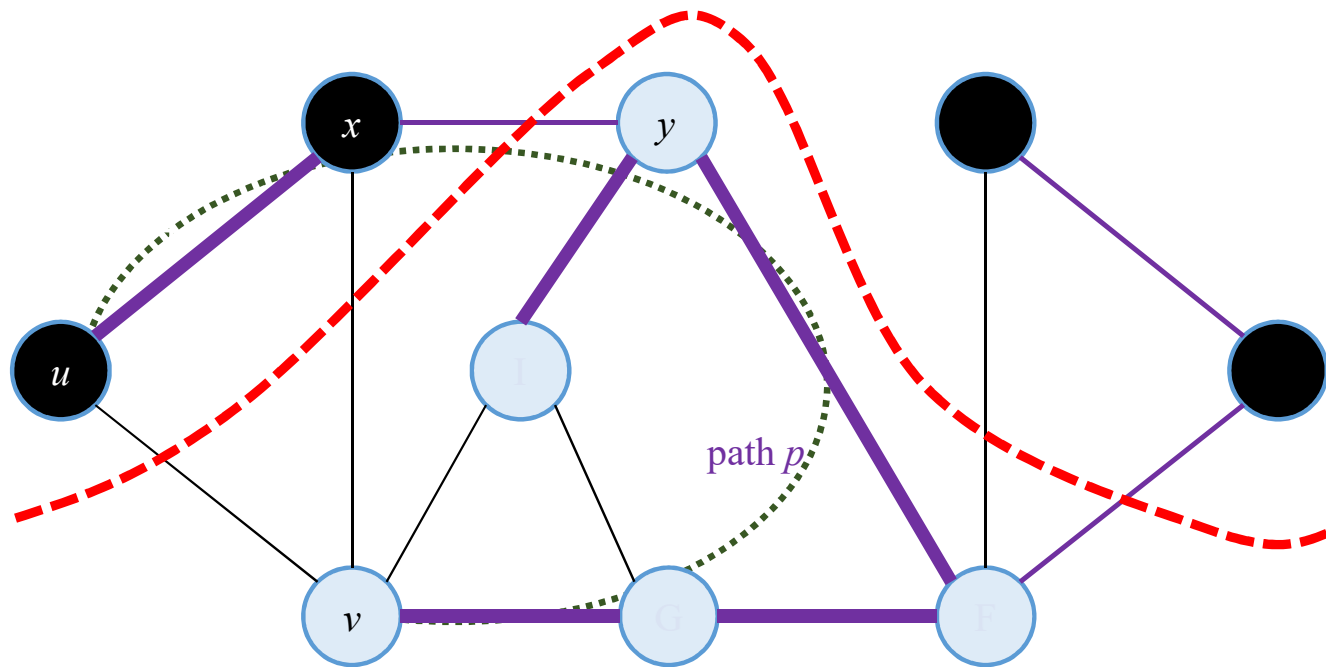
$$T' = T - \{(x, y)\} \cup \{(u, v)\}$$



Proof of Theorem 23.1

$$T' = T - \{(x, y)\} \cup \{(u, v)\}$$

We prove that T' is an MST



Proof of Theorem 23.1

$$T' = T - \{(x, y)\} \cup \{(u, v)\}$$

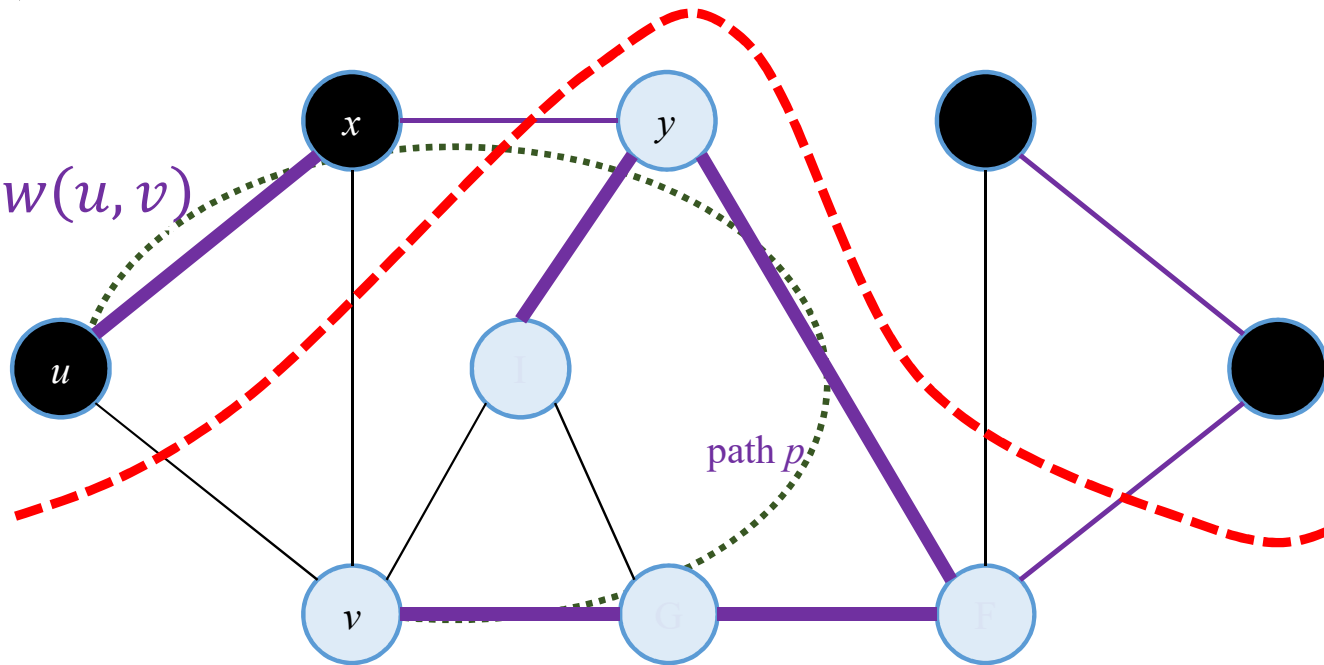
We prove that T' is an MST

Because, (u, v) is a light edge,

$$w(u, v) \leq w(x, y)$$

Now,

$$\begin{aligned} w(T') &= w(T) - w(x, y) + w(u, v) \\ &\leq w(T) \end{aligned}$$



Proof of Theorem 23.1

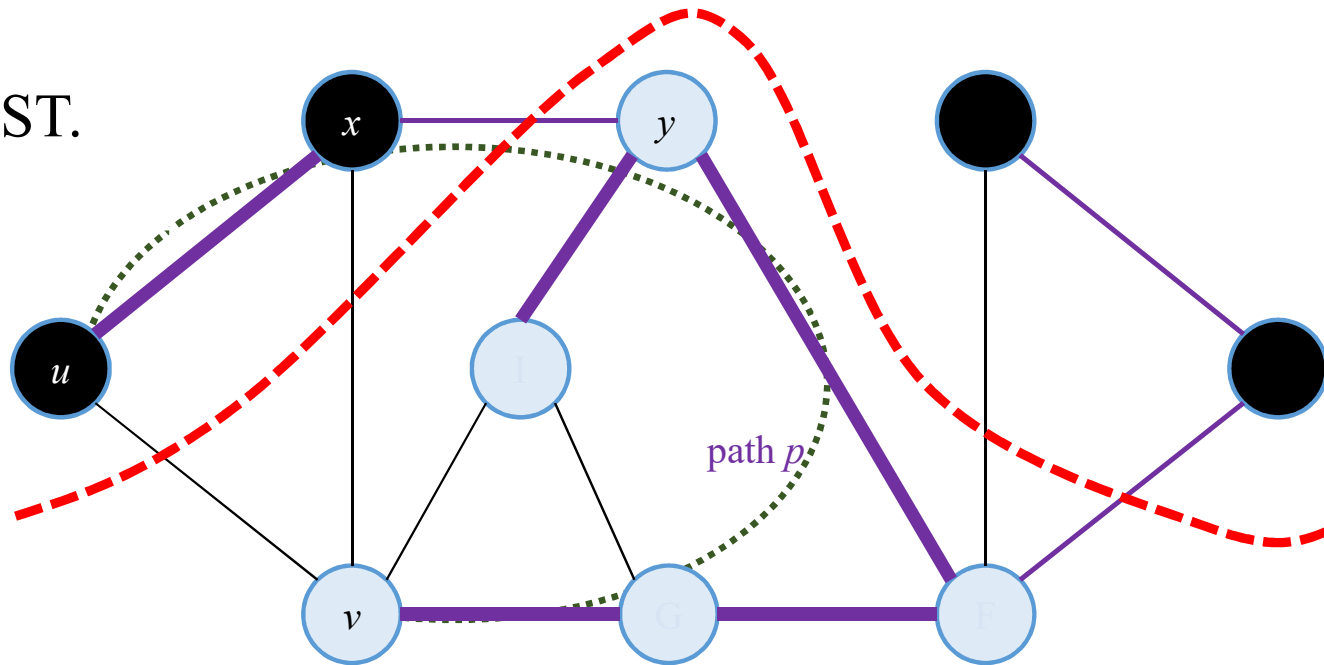
We get,

$$w(T') \leq w(T)$$

but as T is an MST, $w(T) \leq w(T')$

That means, $w(T) = w(T')$

T' is also an MST.

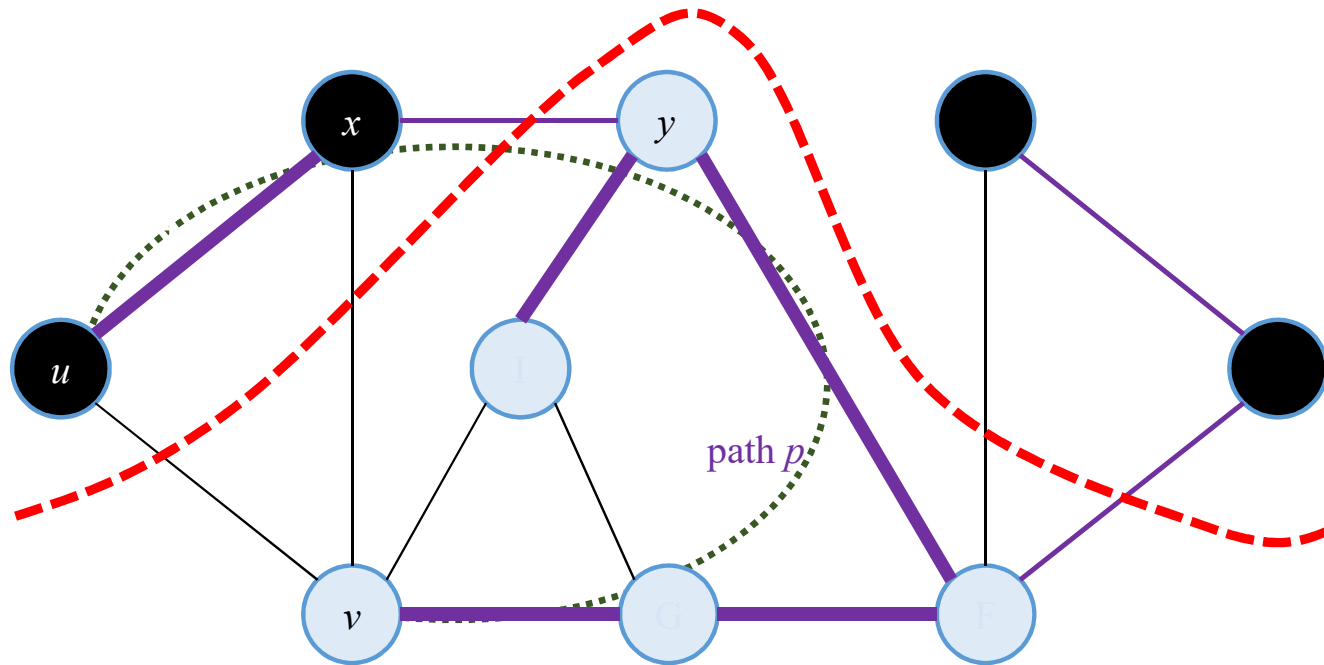


Proof of Theorem 23.1

Now, as $A \subseteq T$, $(x, y) \notin A$, and $T' = T - \{(x, y)\} \cup \{(u, v)\}$

So $A \cup \{(u, v)\} \subseteq T'$

(u, v) is a safe edge for A .



Finding Minimum Spanning Tree

GENERIC-MST(G, w)

```
1   $A = \emptyset$ 
2  while  $A$  does not form a spanning tree
3      find an edge  $(u, v)$  that is safe for  $A$ 
4       $A = A \cup \{(u, v)\}$ 
5  return  $A$ 
```

A grows gradually.

A is always acyclic.

The graph $G_A = (V, A)$ is a forest

Finding Minimum Spanning Tree

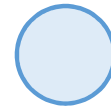
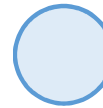
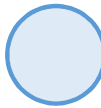
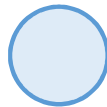
GENERIC-MST(G, w)

```
1   $A = \emptyset$ 
2  while  $A$  does not form a spanning tree
3      find an edge  $(u, v)$  that is safe for  $A$ 
4       $A = A \cup \{(u, v)\}$ 
5  return  $A$ 
```

The graph $G_A = (V, A)$ is a forest

Initially, A is empty

G_A has $|V|$ single node trees.



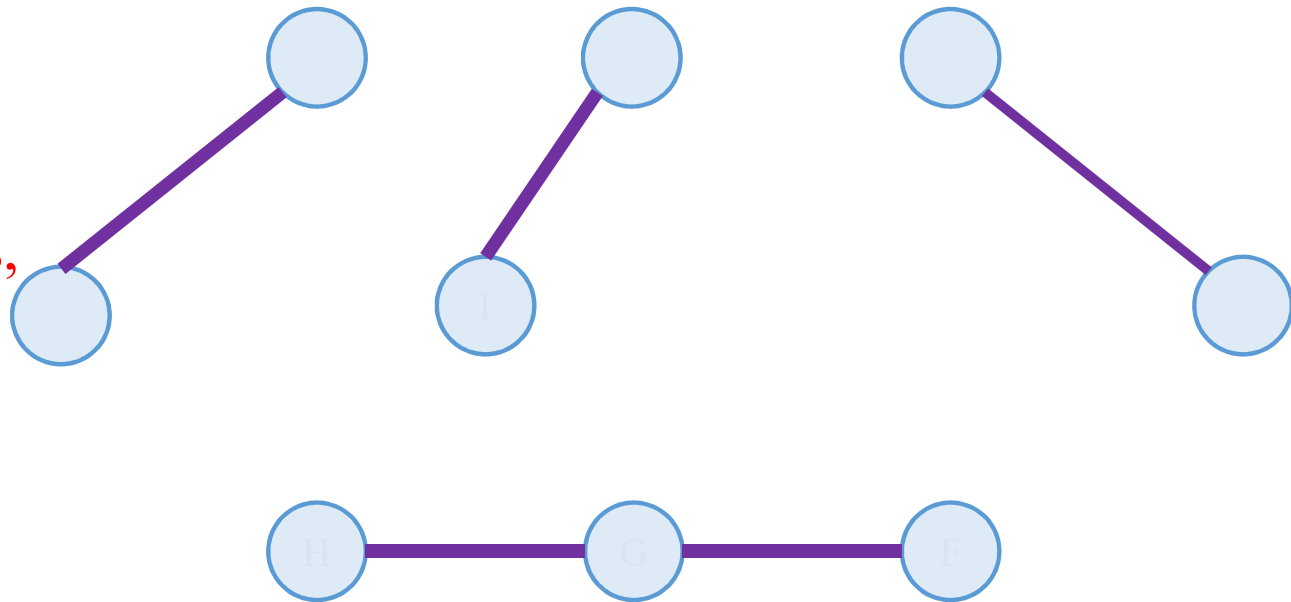
Finding Minimum Spanning Tree

GENERIC-MST(G, w)

```
1   $A = \emptyset$ 
2  while  $A$  does not form a spanning tree
3      find an edge  $(u, v)$  that is safe for  $A$ 
4       $A = A \cup \{(u, v)\}$ 
5  return  $A$ 
```

As A grows by adding edges,

- A must be acyclic
- Produces connected components



Finding Minimum Spanning Tree

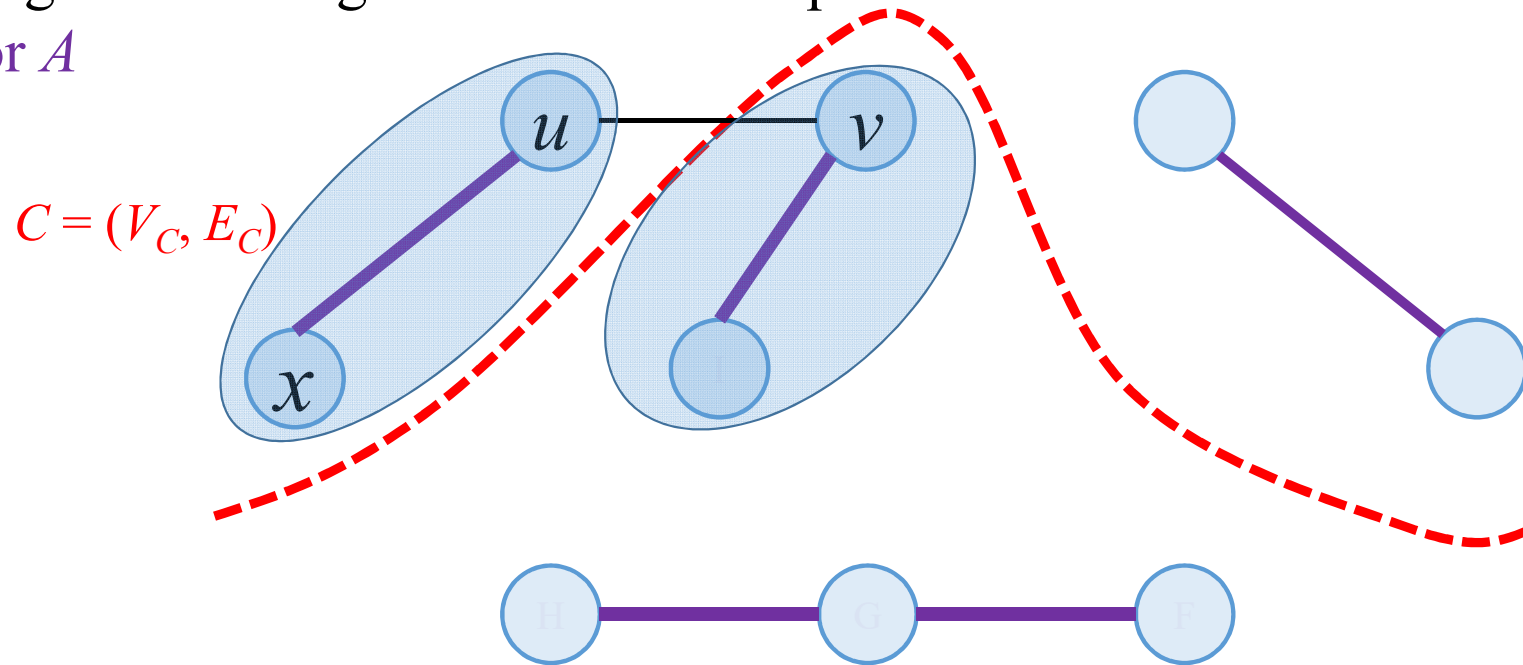
Corollary 23.2:

Let A be a set of edges, and member of an MST.

$C = (V_C, E_C)$ be connected component in forest $G_A = (V, A)$

(u, v) be light edge connecting C to another component.

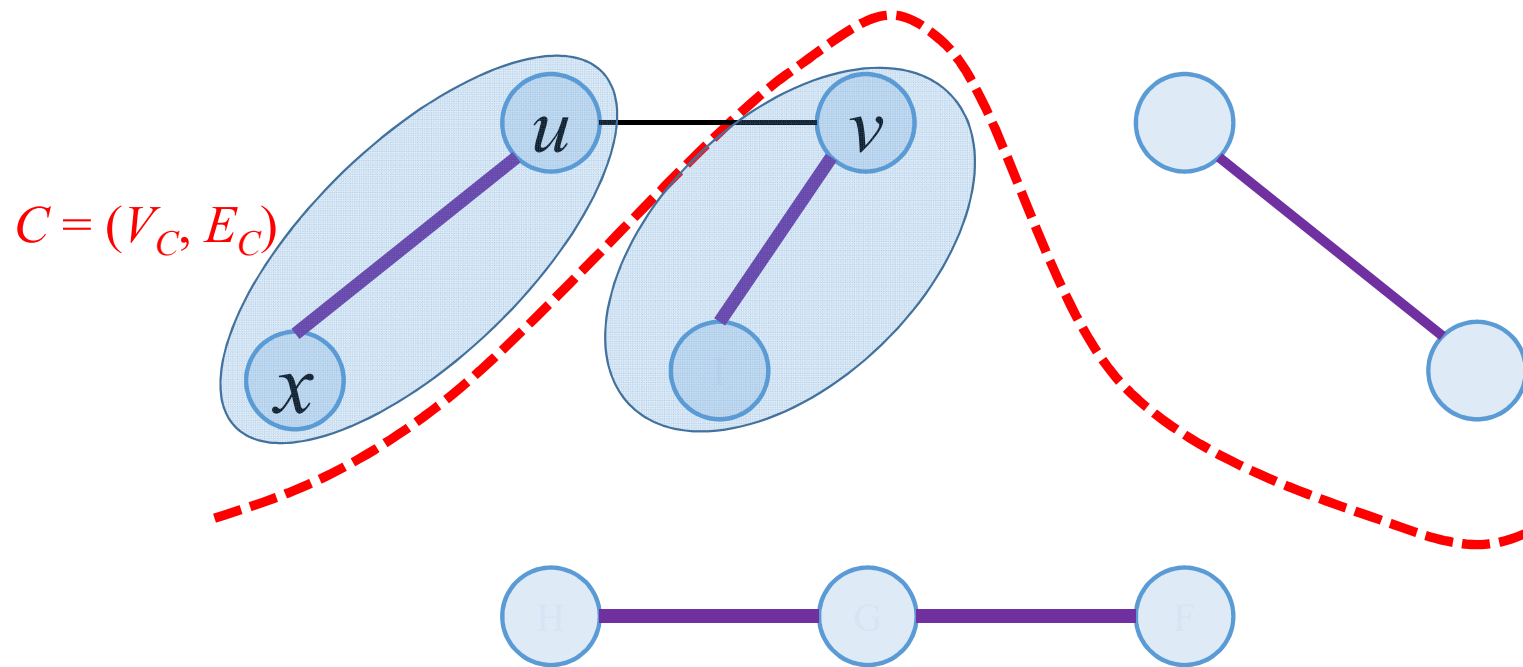
Then (u, v) is safe for A



Proof of 23.2

The Cut $(V_C, V - V_C)$ respects A and (u, v) is a light edge for the cut

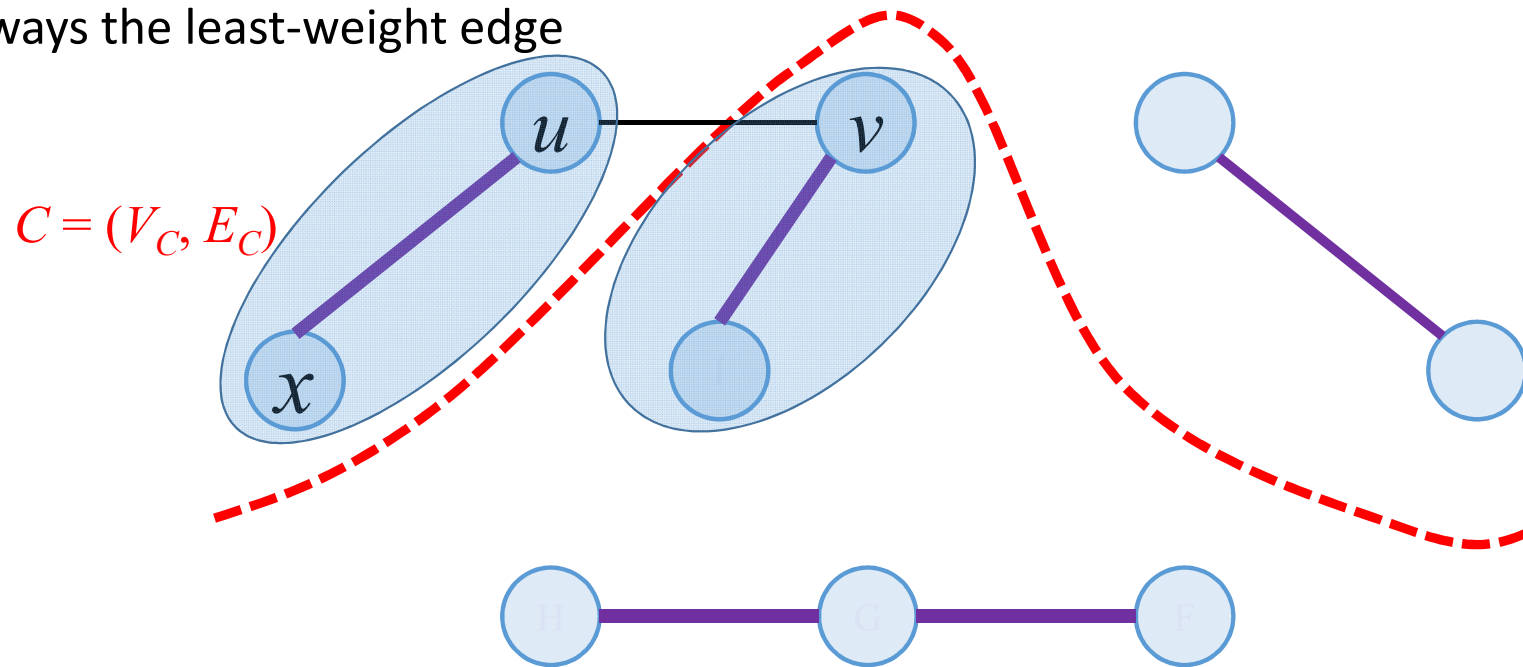
Therefore, (u, v) is safe for A by *Theorem 23.1*



Two Algorithms for MST

- 1st strategy:

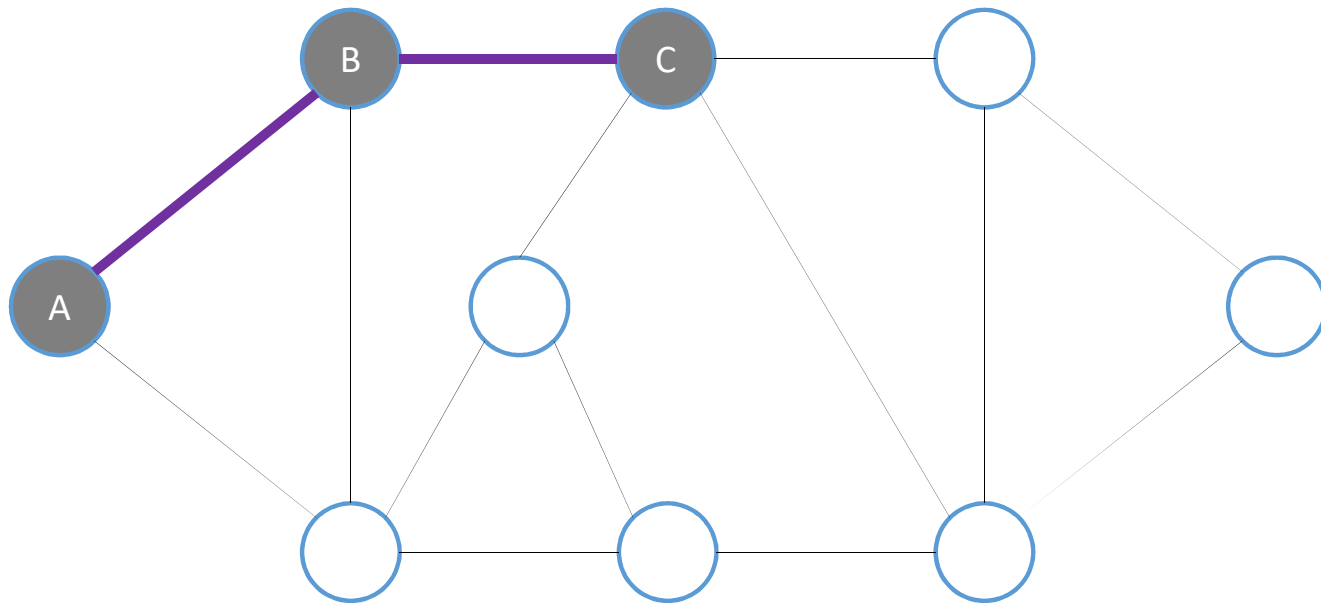
- A is a forest
- Keep adding safe edges to connect components, finally make a single tree
- Safe edge is always the least-weight edge



Two Algorithms for MST

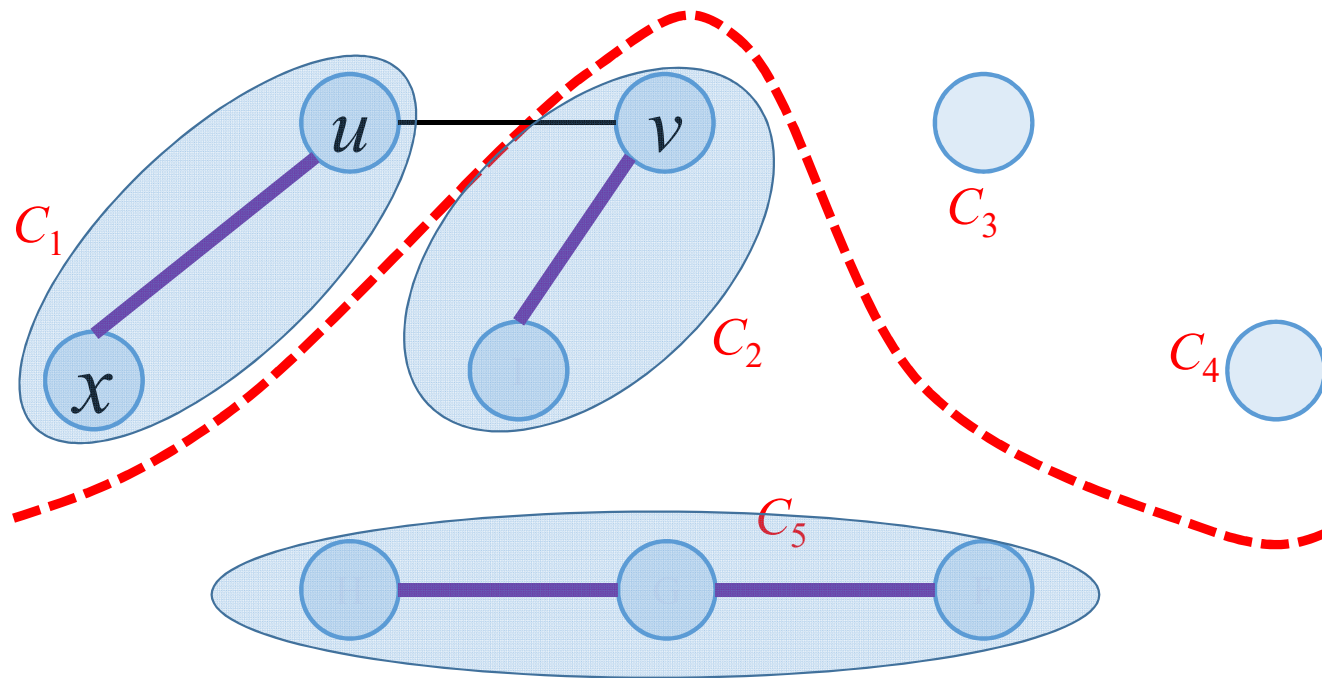
- 2nd strategy:

- Begin with a single tree A , initially a single node
- Keep adding safe edges to A to connect other nodes to A
- Safe edge is always the least-weight edge



Two Algorithms for MST

- 1st strategy: **Kruskal's Algorithm**
 - Connects two components by an safe edge, least-weight edge
 - Light edge (u, v) connects C_1 and C_2
 - Corollary 23.2 guarantees that (u, v) is safe for A

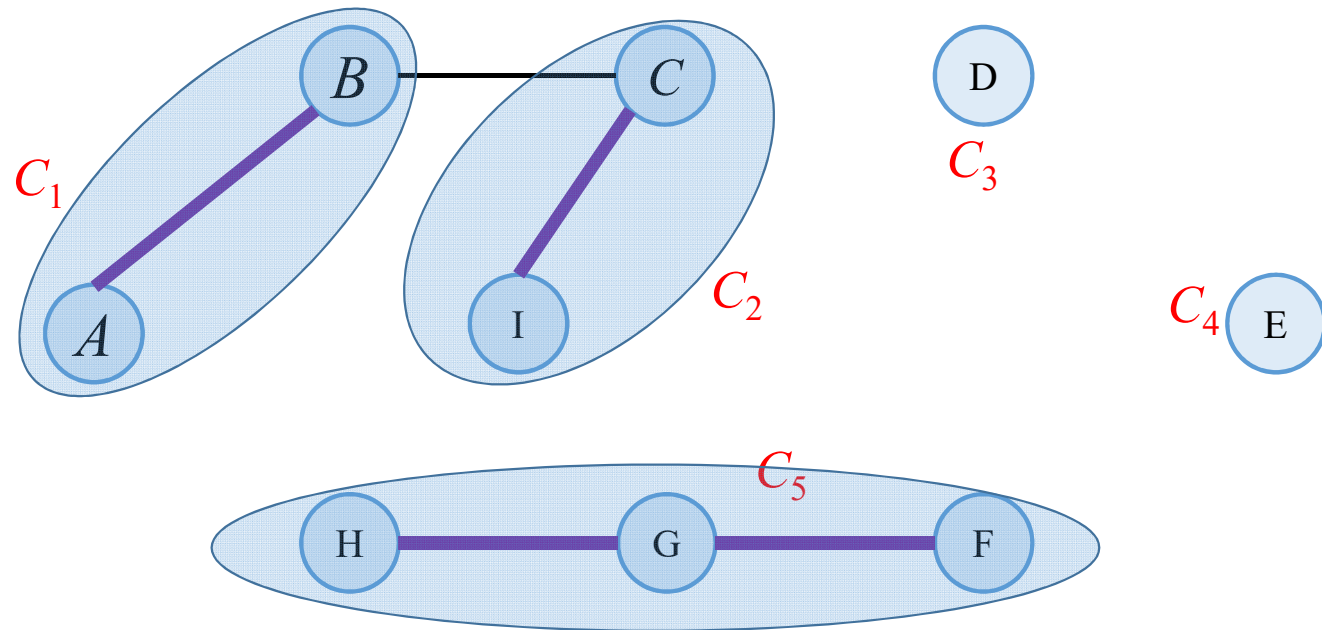


Kruskal's Algorithm for MST

- Sort the edges in E by non-decreasing weight
- $A = \{\}$
- For each edge (u, v) in E
 - If adding (u, v) does not create cycle in A
then $A = A + \{(u, v)\}$
- Return A

Implementation of Kruskal's Algorithm

- Management of nodes of the trees (components)
- Determining if a cycle forms



Implementation of Kruskal's Algorithm

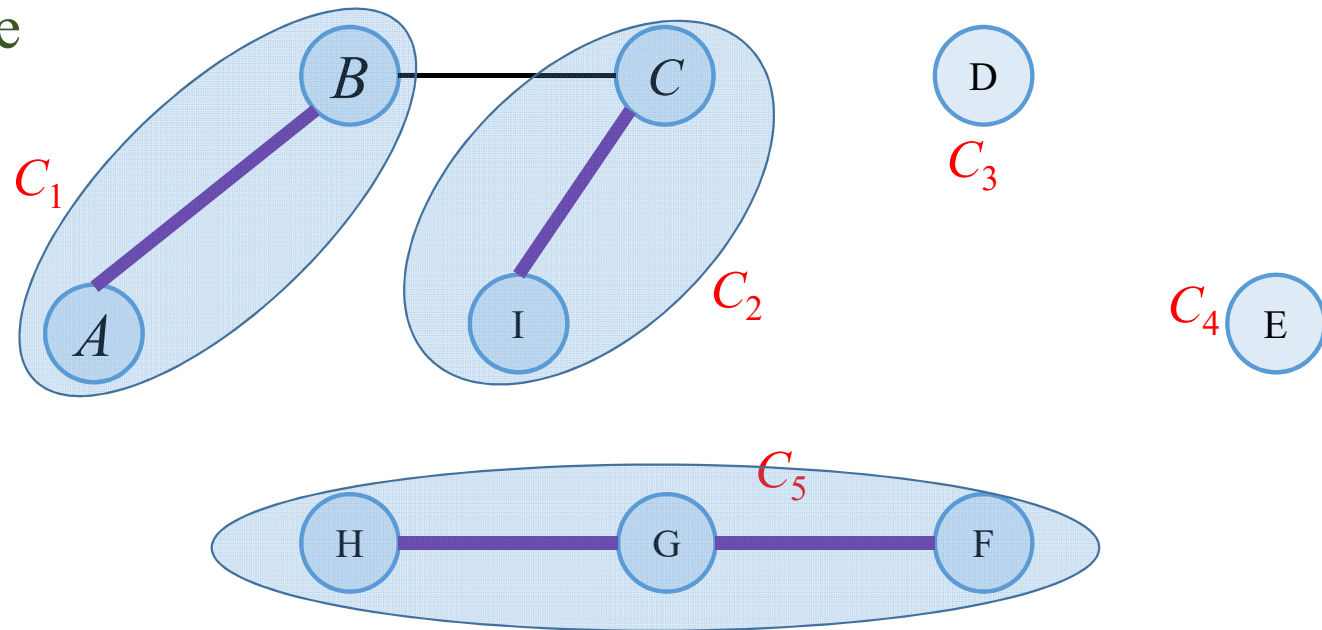
- Management of nodes of the trees (components)
- Determining if a cycle forms

Solution:

- Use disjoint set for each tree

Example:

$\{A, B\}, \{C, I\},$
 $\{H, G, F\}, \{D\}, \{E\}$



Implementation of Kruskal's Algorithm

- Management of nodes of the trees (components)
- Determining if a cycle forms

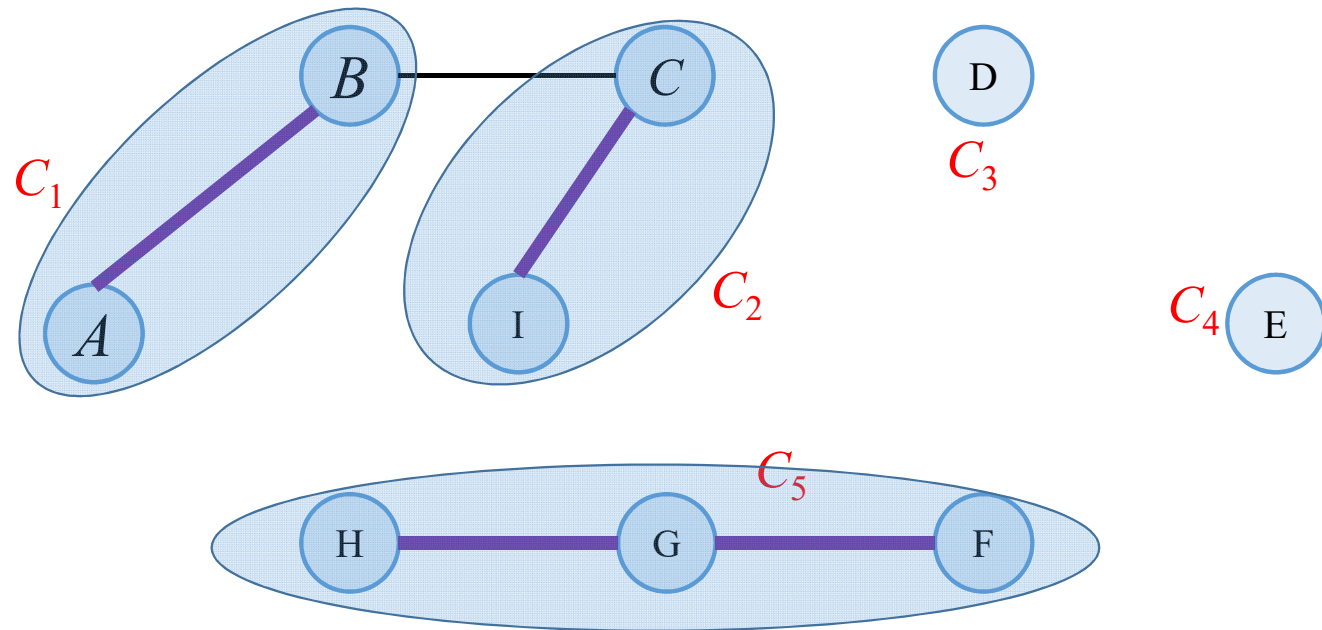
Solution:

- Use disjoint set for each tree

Example:

$\{A, B\}$, $\{C, I\}$,
 $\{H, G, F\}$, $\{D\}$, $\{E\}$

- If an edge has endpoints in same set, it will create cycle



Kruskal's Algorithm for MST

- Sort the edges in E by non-decreasing weight
- $A = \{\}$
- For each edge (u, v) in E
 - If adding (u, v) does not create cycle in A
then $A = A + \{(u, v)\}$
- Return A

MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

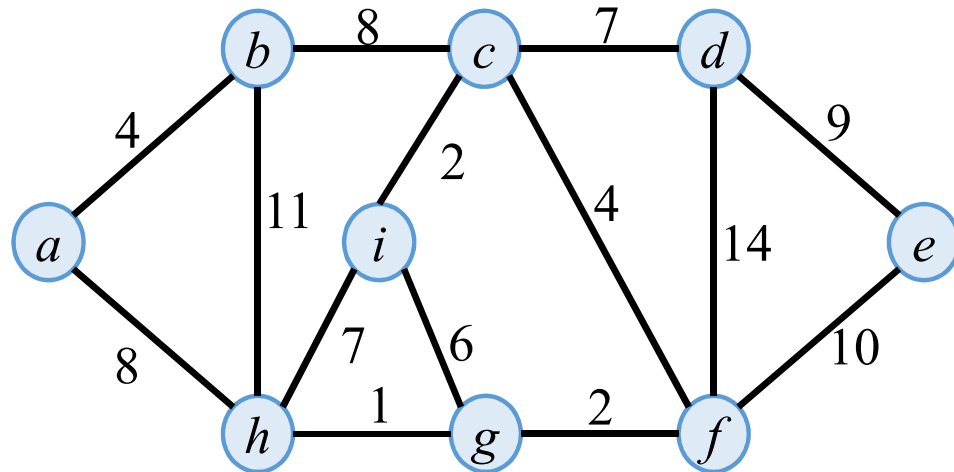
Simulation of Kruskal's Algorithm

MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

Sets:

$\{a\}, \{b\}, \{c\}, \{d\}, \{e\}, \{f\},$
 $\{g\}, \{h\}, \{i\}$



Simulation of Kruskal's Algorithm

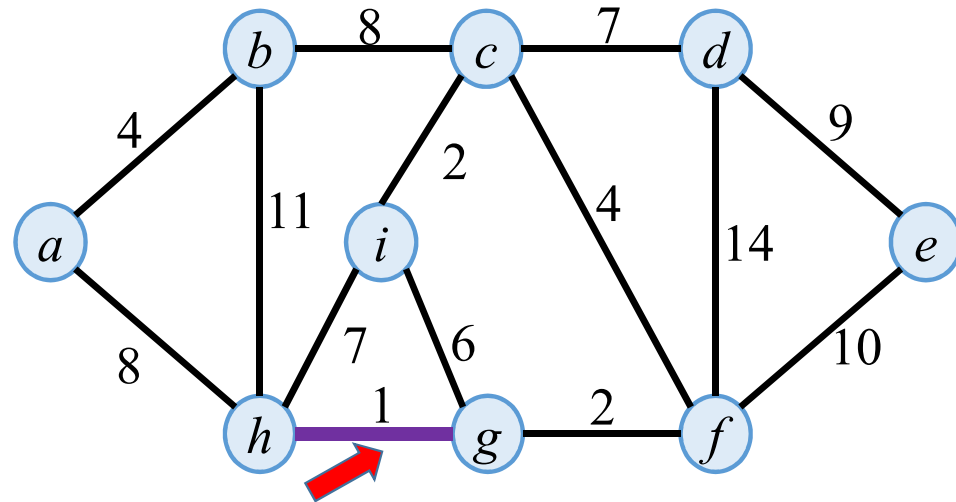
MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

Sets:

$\{a\}, \{b\}, \{c\}, \{d\}, \{e\}, \{f\},$

$\{g, h\}, \{i\}$



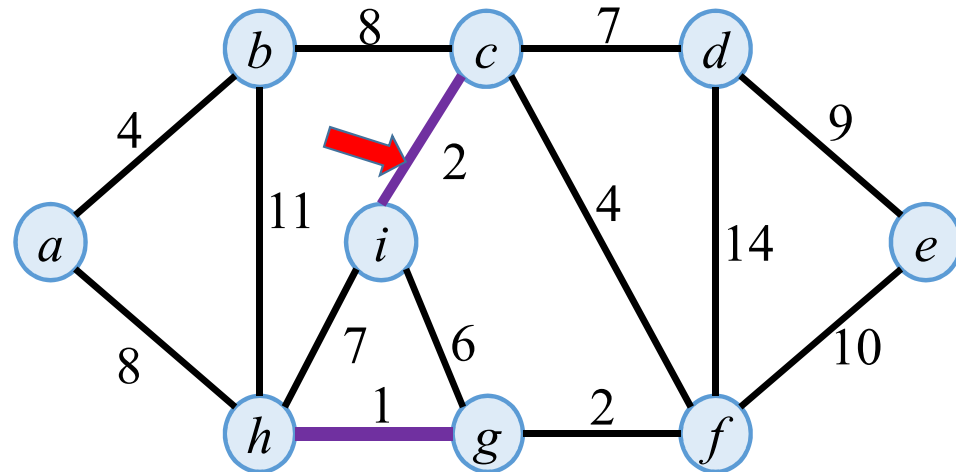
Simulation of Kruskal's Algorithm

MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

Sets:

$\{a\}, \{b\}, \{c, i\}, \{d\}, \{e\}, \{f\},$
 $\{g, h\}, \{h\}, \{i\}$



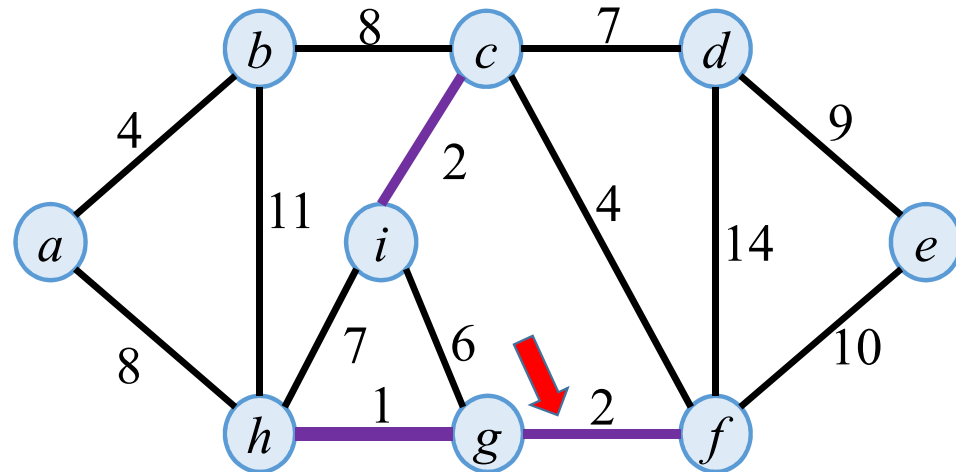
Simulation of Kruskal's Algorithm

MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

Sets:

$\{a\}, \{b\}, \{c, i\}, \{d\}, \{e\}, \{f\},$
 $\{f, g, h\}, \{h\}, \{i\}$



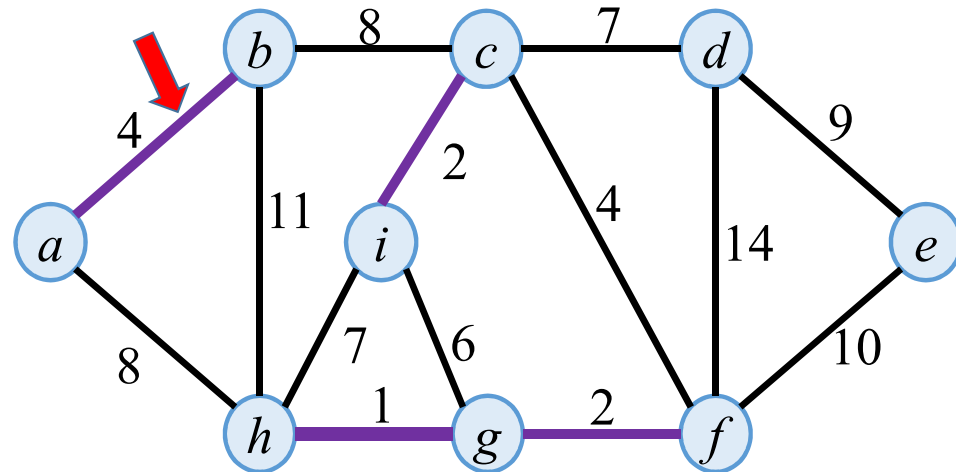
Simulation of Kruskal's Algorithm

MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

Sets:

$\{a, b\}$, ~~$\{b\}$~~ , $\{c, i\}$, $\{d\}$, $\{e\}$, ~~$\{f\}$~~ ,
 $\{f, g, h\}$, ~~$\{h\}$~~ , ~~$\{i\}$~~



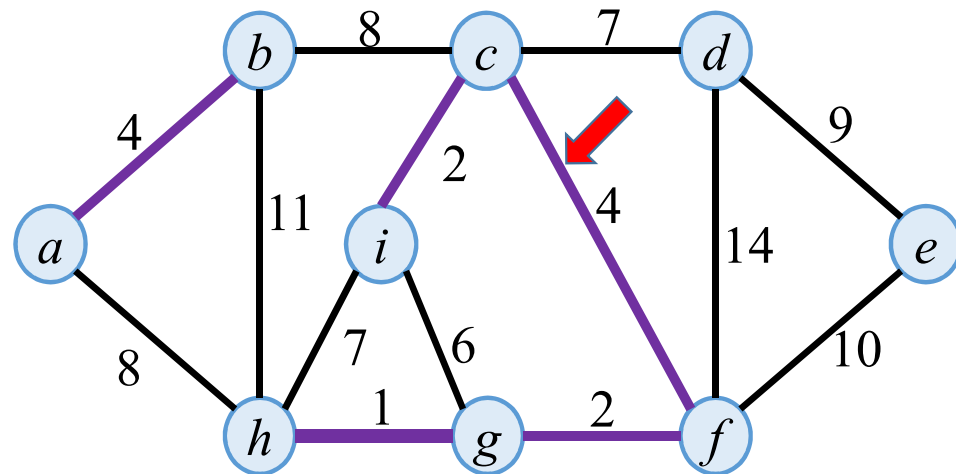
Simulation of Kruskal's Algorithm

MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

Sets:

$\{a, b\}, \{b\}, \{c, i\}, \{d\}, \{e\}, \{f\},$
 $\{c, f, g, h, i\}, \{h\}, \{i\}$



Simulation of Kruskal's Algorithm

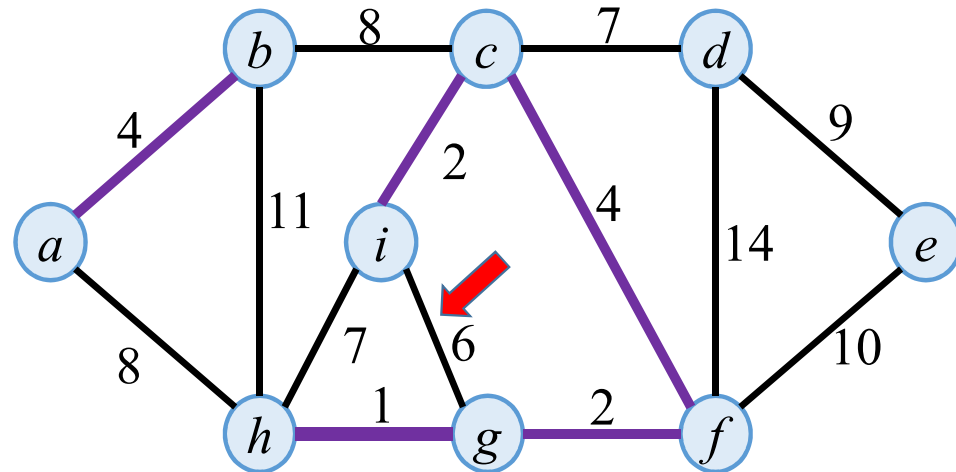
MST-KRUSKAL(G, w)

```

1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
    
```

Sets:

$\{a, b\}, \{b\}, \{c, i\}, \{d\}, \{e\}, \{f\},$
 $\{c, f, g, h, i\}, \{h\}, \{i\}$



Simulation of Kruskal's Algorithm

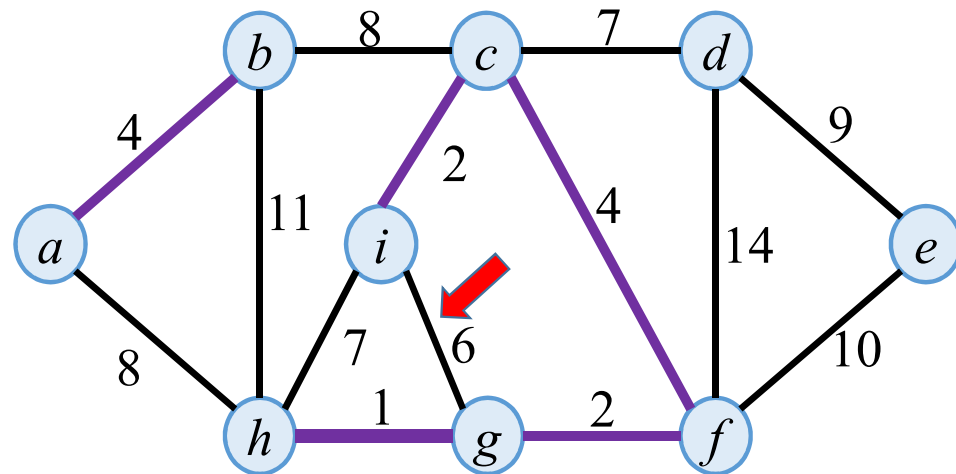
MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

Sets:

$\{a, b\}, \{b\}, \{c, i\}, \{d\}, \{e\}, \{f\},$
 $\{c, f, g, h, i\}, \{h\}, \{i\}$

Edge (i, g) will
create a cycle
So, we discard it



Simulation of Kruskal's Algorithm

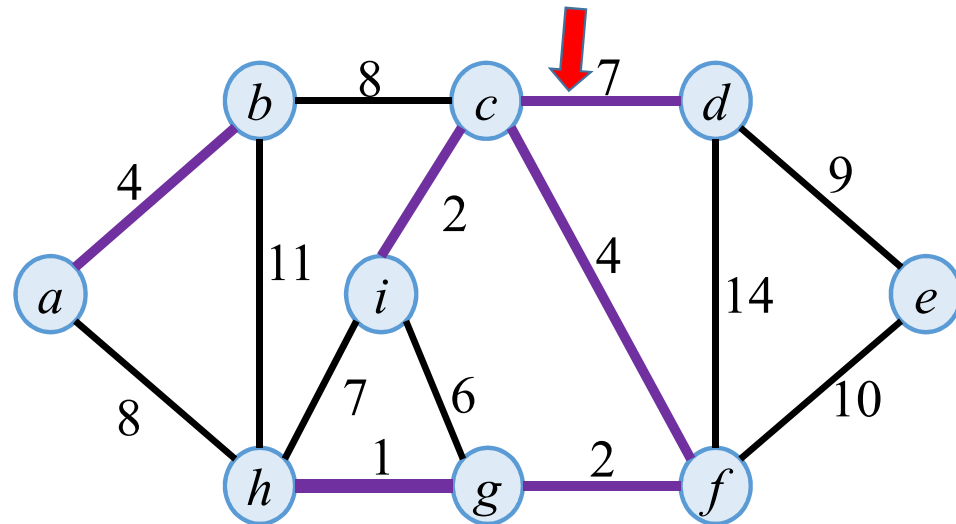
MST-KRUSKAL(G, w)

```

1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
    
```

Sets:

$\{a, b\}, \{b\}, \{c, i\}, \{d\}, \{e\}, \{f\},$
 $\{c, d, f, g, h, i\}, \{h\}, \{i\}$



Simulation of Kruskal's Algorithm

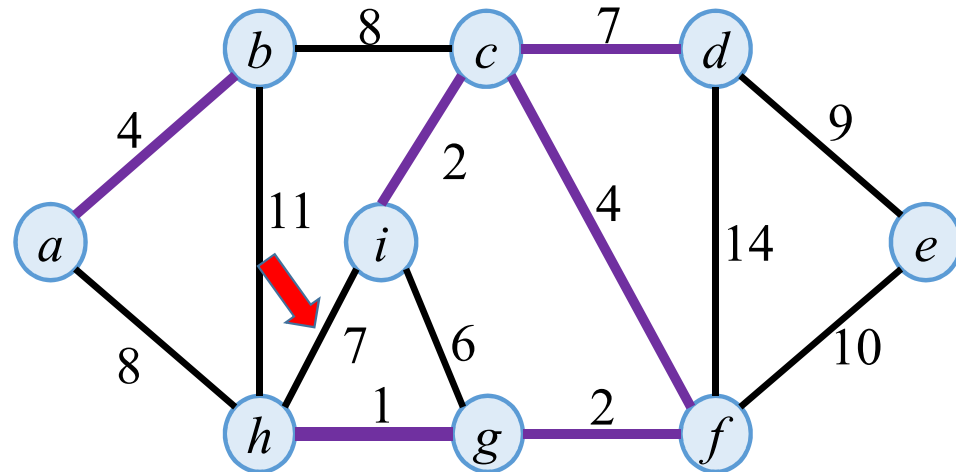
MST-KRUSKAL(G, w)

```

1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
    
```

Sets:

$\{a, b\}, \{b\}, \{c, i\}, \{d\}, \{e\}, \{f\},$
 $\{c, d, f, g, h, i\}, \{h\}, \{i\}$



Simulation of Kruskal's Algorithm

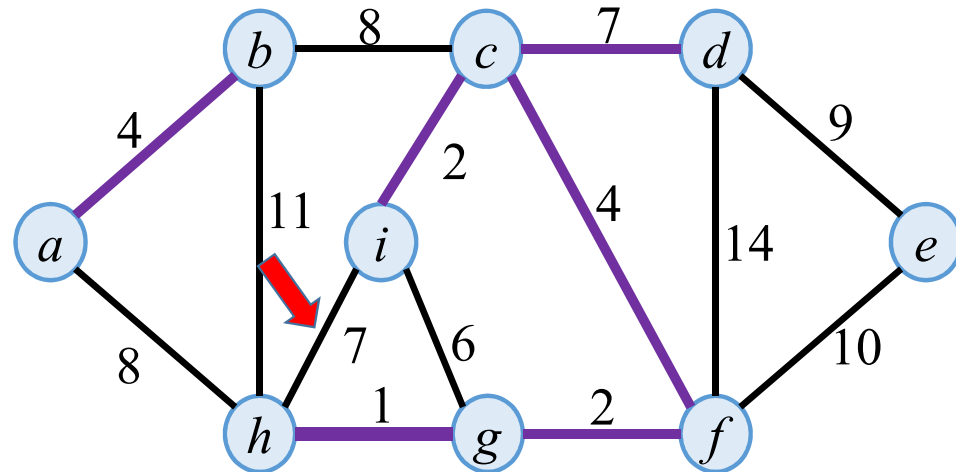
MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

Sets:

$\{a, b\}$, $\{b\}$, $\{c, i\}$, $\{d\}$, $\{e\}$, $\{f\}$,
 $\{c, d, f, g, h, i\}$, $\{h\}$, $\{i\}$

Edge (h, i) will
create a cycle
So, we discard it



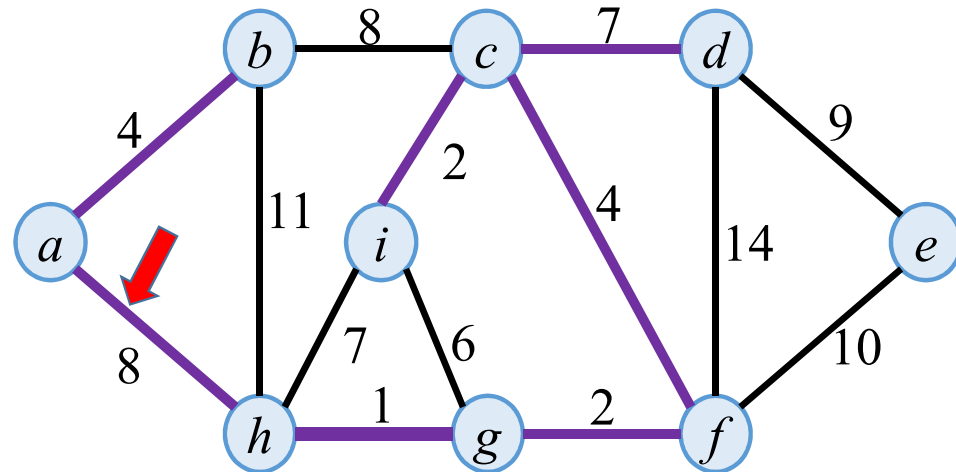
Simulation of Kruskal's Algorithm

MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

Sets:

~~$\{a, b\}$~~ , ~~$\{b\}$~~ , ~~$\{c, i\}$~~ , ~~$\{d\}$~~ , ~~$\{e\}$~~ , ~~$\{f\}$~~ ,
 ~~$\{a, b, c, d, f, g, h, i\}$~~ , ~~$\{h\}$~~ , ~~$\{i\}$~~



Simulation of Kruskal's Algorithm

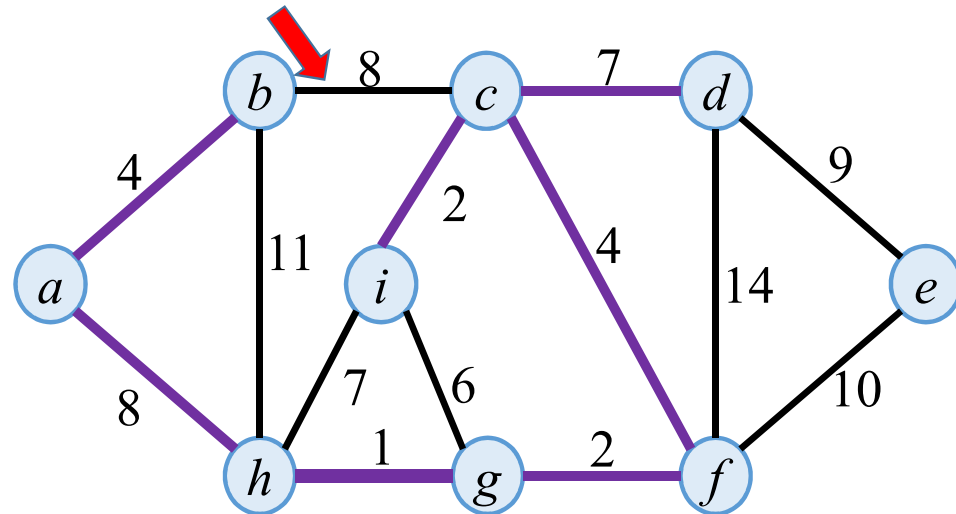
MST-KRUSKAL(G, w)

```

1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
    
```

Sets:

~~$\{a, b\}$~~ , ~~$\{b\}$~~ , ~~$\{c, i\}$~~ , ~~$\{d\}$~~ , ~~$\{e\}$~~ , ~~$\{f\}$~~ ,
 ~~$\{a, b, c, d, f, g, h, i\}$~~ , ~~$\{h\}$~~ , ~~$\{i\}$~~



Simulation of Kruskal's Algorithm

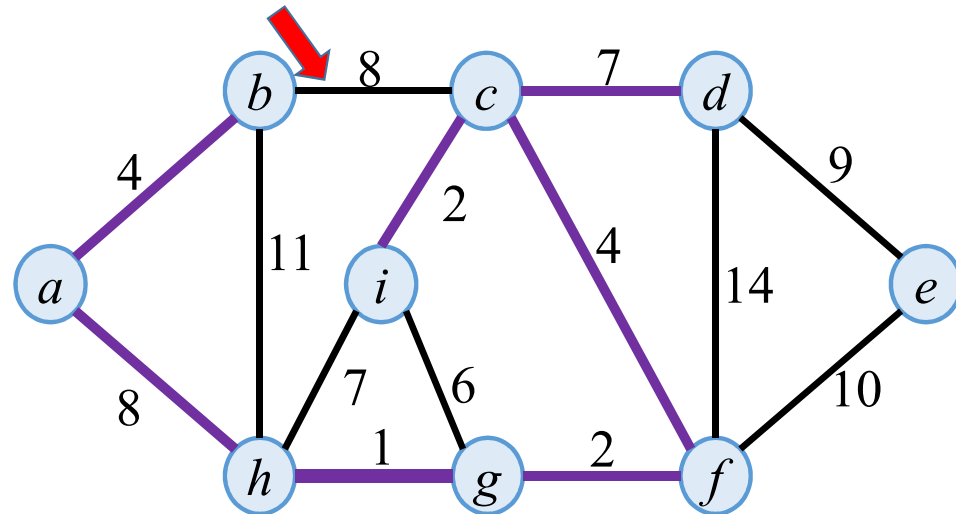
MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

Sets:

~~$\{a, b\}$~~ , ~~$\{b\}$~~ , ~~$\{c, i\}$~~ , ~~$\{d\}$~~ , ~~$\{e\}$~~ , ~~$\{f\}$~~ ,
 ~~$\{a, b, c, d, f, g, h, i\}$~~ , ~~$\{h\}$~~ , ~~$\{i\}$~~

Edge (b, c) will
create a cycle
So, we discard it



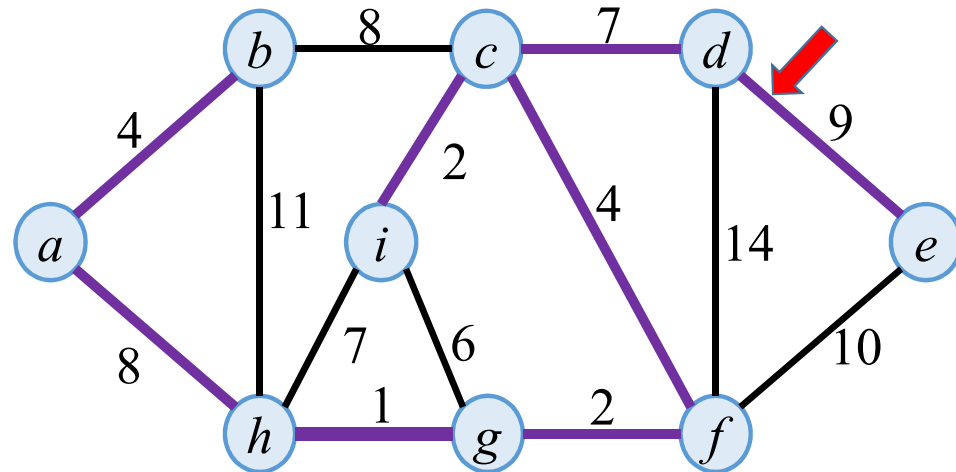
Simulation of Kruskal's Algorithm

MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

Sets:

~~$\{a, b\}$~~ , ~~$\{b\}$~~ , ~~$\{c, i\}$~~ , ~~$\{d\}$~~ , ~~$\{e\}$~~ , ~~$\{f\}$~~ ,
 ~~$\{a, b, c, d, e, f, g, h, i\}$~~ , ~~$\{h\}$~~ , ~~$\{i\}$~~



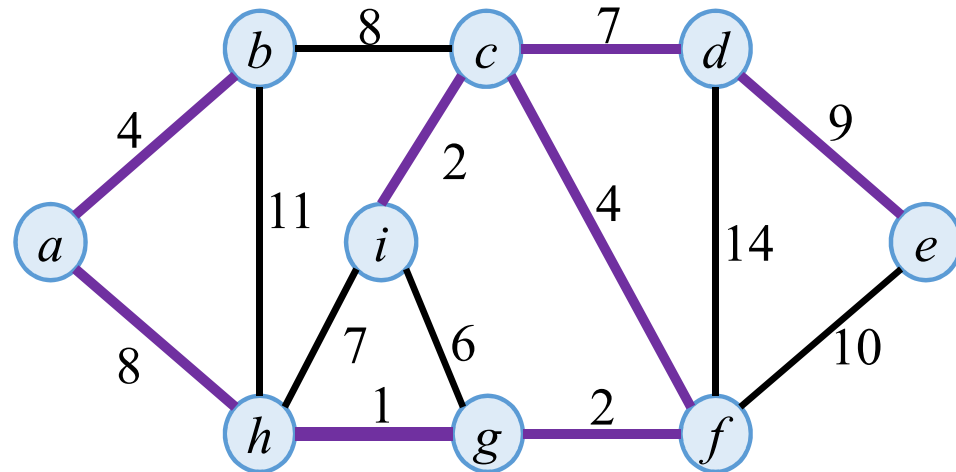
Simulation of Kruskal's Algorithm

MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

Sets:

$\{a, b, c, d, e, f, g, h, i\}$



Simulation of Kruskal's Algorithm

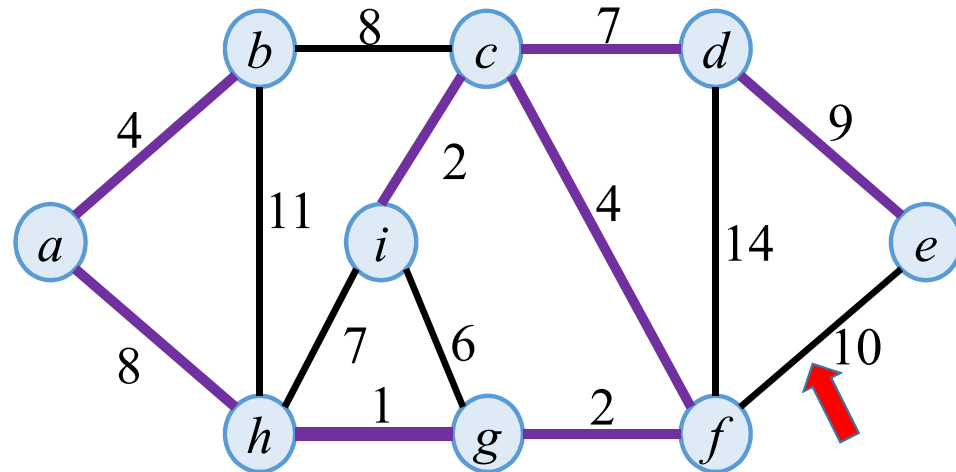
MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

Sets:

$\{a, b, c, d, e, f, g, h, i\}$

Edge (e, f) will
create a cycle
So, we discard it



Simulation of Kruskal's Algorithm

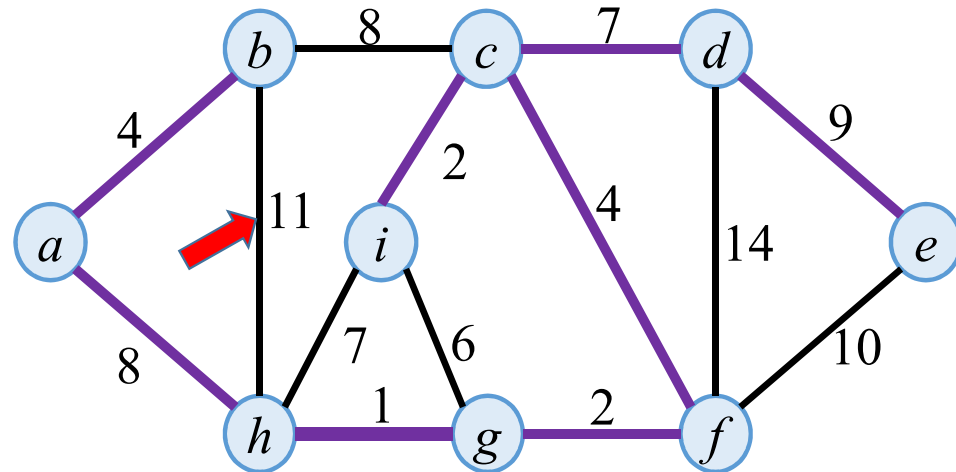
MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

Sets:

$\{a, b, c, d, e, f, g, h, i\}$

Edge (b, h) will
create a cycle
So, we discard it



Simulation of Kruskal's Algorithm

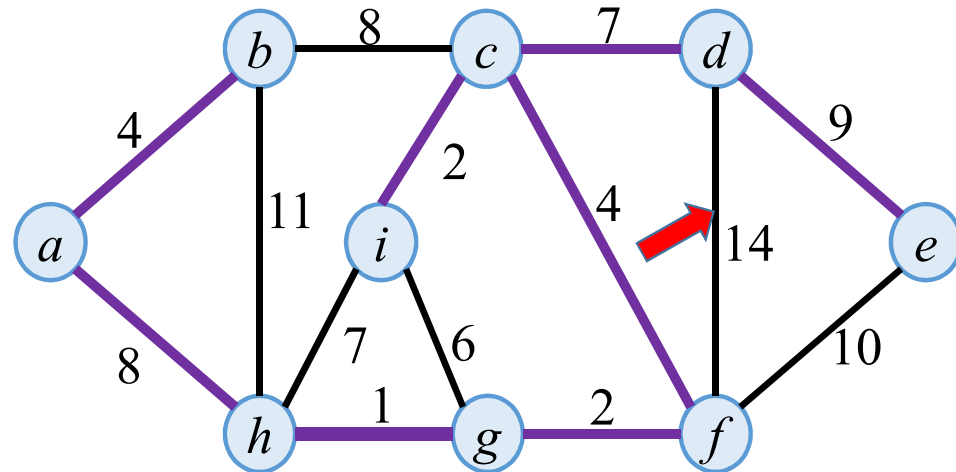
MST-KRUSKAL(G, w)

```
1   $A = \emptyset$ 
2  for each vertex  $v \in G.V$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

Sets:

$\{a, b, c, d, e, f, g, h, i\}$

Edge (d, f) will
create a cycle
So, we discard it



Complexity of Kruskal's Algorithm

MST-KRUSKAL(G, w)

```
1   $A = \emptyset \rightarrow O(1)$ 
2  for each vertex  $v \in G.V \rightarrow O(V)$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w \rightarrow O(E \lg E)$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight  $O(E)$ 
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

Complexity of Kruskal's Algorithm

MST-KRUSKAL(G, w)

```
1   $A = \emptyset \rightarrow O(1)$ 
2  for each vertex  $v \in G.V \rightarrow O(V)$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w \rightarrow O(E \lg E)$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight  $O(E)$ 
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

Make-Set, Find-Set and union *takes* $\alpha(V)$

All set operations together take $O((V + E)\alpha(V))$

Complexity of Kruskal's Algorithm

MST-KRUSKAL(G, w)

```
1   $A = \emptyset \rightarrow O(1)$ 
2  for each vertex  $v \in G.V \rightarrow O(V)$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w \rightarrow O(E \lg E)$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight  $O(E)$ 
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

All set operations together take $O((V + E)\alpha(V))$

$\alpha(V)$ is slowly varying component = $\lg(V)$

Complexity of Kruskal's Algorithm

MST-KRUSKAL(G, w)

```
1   $A = \emptyset \rightarrow O(1)$ 
2  for each vertex  $v \in G.V \rightarrow O(V)$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w \rightarrow O(E \lg E)$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight  $O(E)$ 
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

The graph is connected, $|E| \geq |V| - 1$

Maximum No. of edges = $|E| < |V|^2$

Complexity of Kruskal's Algorithm

MST-KRUSKAL(G, w)

```
1   $A = \emptyset \rightarrow O(1)$ 
2  for each vertex  $v \in G.V \rightarrow O(V)$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w \rightarrow O(E \lg E)$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight  $O(E)$ 
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

$$|E| \geq |V| - 1 \text{ and } |E| < |V|^2$$

That means, $O(\lg(V)) = O(\lg(E))$

Complexity of Kruskal's Algorithm

MST-KRUSKAL(G, w)

```
1   $A = \emptyset \rightarrow O(1)$ 
2  for each vertex  $v \in G.V \rightarrow O(V)$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w \rightarrow O(E \lg E)$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight  $O(E)$ 
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

$$O(\lg(V)) = O(\lg(E))$$

All set operations together take $O(E \alpha(V)) = O(E \lg(V)) = O(E \lg(E))$

Complexity of Kruskal's Algorithm

MST-KRUSKAL(G, w)

```
1   $A = \emptyset \rightarrow O(1)$ 
2  for each vertex  $v \in G.V \rightarrow O(V)$ 
3      MAKE-SET( $v$ )
4  sort the edges of  $G.E$  into nondecreasing order by weight  $w \rightarrow O(E \lg E)$ 
5  for each edge  $(u, v) \in G.E$ , taken in nondecreasing order by weight  $O(E)$ 
6      if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
7           $A = A \cup \{(u, v)\}$ 
8          UNION( $u, v$ )
9  return  $A$ 
```

Total running time = set operations + sorting = $O(E \lg(E)) = O(E \lg(V))$

Two Algorithms for MST

- 2nd strategy:

- Begin with single tree A , initially a single node
- Keep adding safe edges to A to connect other nodes to A
- Safe edge is always the least-weight edge

